LANGGRAPH TRACK

AI 에이전트 개발 심화

LangGraph로 상태와 흐름 제어하기

주식회사 똑똑한청년들 · 배움 에이아이

PART 01

오리엔테이션

입문과 LangChain 기초에서 감춰져 있던 실행 흐름을 StateGraph와 Functional API로 직접 제어합니다.

LangGraph는 에이전트의 흐름을 코드가 아니라 그래프로 고정합니다

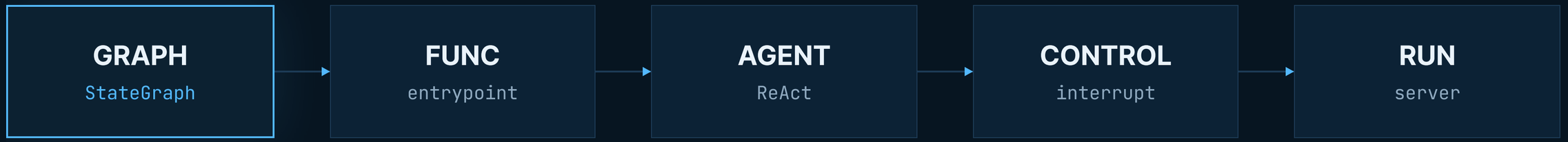
상태·노드·엣지를 명시하면 분기, 반복, 중단, 재개를 같은 모델로 설명할 수 있습니다

State 공유 데이터	노드들이 읽고 갱신하는 TypedDict·MessagesState입니다. 리듀서가 누적 방식을 정합니다.	`TypedDict · Annotated`
Node 작업 단위	상태 일부를 받아 바뀐 부분만 반환하는 Python 함수입니다. LLM 호출도 하나의 노드가 됩니다.	`def node(state) → dict`
Edge 흐름 제어	고정 연결과 조건부 연결로 다음 노드를 결정합니다. START와 END가 실행 경계를 만듭니다.	`add_edge · add_conditional_edges`

© 주식회사 똑똑한청년들 · 배움 에이아이

이번 트랙은 그래프 작성에서 운영 배포까지 한 번에 이어집니다

각 노트북은 LangGraph가 제공하는 제어 지점을 하나씩 넓힙니다



GRAPH

상태 스키마와 노드·엣지로 기본 실행 흐름을 만듭니다.

FUNC

함수형 워크플로와 task checkpoint를 배웁니다.

AGENT

도구 호출 루프를 직접 구성합니다.

CONTROL

사람 승인과 타임 트래블로 실행을 조정합니다.

RUN

테스트, Studio, 로컬 서버, 내구성 실행까지 연결합니다.

LangGraph 심화의 핵심은 모델이 아니라 **상태 전이**를 설계하는 능력입니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

03_langgraph는 13개 노트북으로 그래프 제어의 전체 표면을 다룹니다

Graph API와 Functional API를 모두 써 보고, 운영에서 필요한 중단·재개·관측까지 확장합니다

구간	노트북	학습 산출
기본 구조	01~03 소개·Graph API·Functional API	StateGraph와 @entrypoint/@task를 모두 설명할 수 있습니다
패턴과 에이전트	04~05 워크플로·ReAct 에이전트	분기·병렬·오케스트레이터·도구 루프를 직접 작성합니다
상태 제어	06~09 메모리·스트리밍·인터럽트·서브그래프	실행 이력, 사람 승인, replay/fork, namespace 격리를 다룹니다
운영	10~13 프로덕션·서버·내구성·Pregel	테스트, 배포, SDK, fault tolerance, API 선택 기준을 정리합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 02

LangGraph 실행 모델

LangChain 위에서 복잡한 에이전트 흐름을 상태 그래프로 설계하는 이유를 정리합니다.

실습 · 03_langgraph/01_introduction.ipynb

LangGraph는 상태를 중심으로 실행·중단·재개를 설명합니다

입문자는 노드보다 먼저 상태가 어떻게 변하는지 추적해야 합니다

개념	의미	학습 포인트
State	그래프가 읽고 쓰는 공유 데이터	TypedDict와 reducer를 구분합니다
Node	상태를 받아 일부 상태를 반환하는 함수	노드는 전체 상태가 아니라 변경분을 돌려줍니다
Edge	다음 노드를 결정하는 연결	조건부 엣지는 라우팅 함수와 함께 봅니다
Checkpoint	실행 중 상태 저장	interrupt, time travel, memory의 전제입니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Graph API와 Functional API는 표현 방식이 다를 뿐 같은 실행 모델을 공유합니다

초반에 두 API의 역할을 구분하면 뒤의 노트북을 읽기 쉬워집니다

Graph API

상태 스키마, 노드, 엣지를 명시적으로 선언합니다.

`StateGraph`

Functional API

@entrypoint와 @task로 함수 호출처럼 그래프 기능을 사용합니다.

`entrypoint · task`

공통 기반

둘 다 체크포인터, 스트리밍, interrupt 같은 런타임 기능을 사용합니다.

`checkpointer`

선택 기준

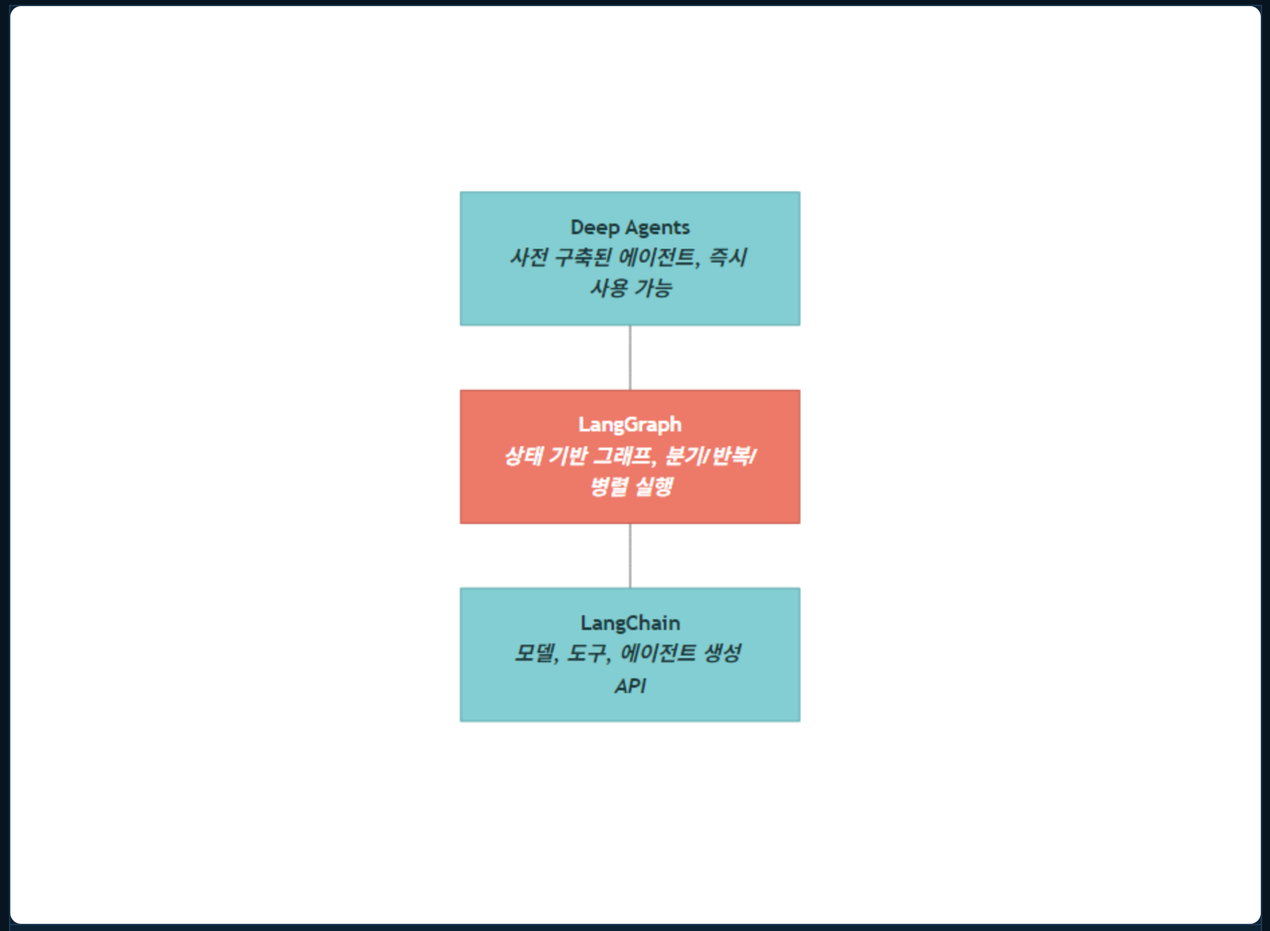
흐름을 문서화해야 하면 Graph API, 코드 흐름이 자연스러우면 Functional API가 편합니다.

`API choice`

© 주식회사 똑똑한청년들 · 배움 에이아이

LangChain·LangGraph·Deep Agents는 계층이 다르고 책임도 다릅니다

LangGraph는 LangChain의 모델·도구 인터페이스 위에 상태 기반 오케스트레이션을 올립니다



LangChain 3계층

LangChain

모델과 도구를 연결하는 기본 인터페이스입니다.

LangGraph

상태·노드·엣지로 실행 흐름을 직접 제어합니다.

Deep Agents

LangGraph 위에 파일·서브에이전트·메모리·스킬을 얹는 장기 작업 하네스입니다. 계획은 필요할 때 TodoListMiddleware로 추가합니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

Graph API와 Functional API는 같은 런타임을 다른 표면으로 다룹니다

복잡한 흐름은 Graph API, 직선형 파이프라인은 Functional API가 읽기 쉽습니다

API	작성 방식	적합한 상황
Graph API	StateGraph에 노드와 엣지를 명시	분기·반복·시각화·복잡한 에이전트 제어
Functional API	@entrypoint와 @task로 함수형 작성	작업 파이프라인·병렬 태스크·짧은 워크플로
공통점	checkpointer와 stream을 공유	중단·재개·관측을 같은 방식으로 다룸

StateGraph의 최소 실행 흐름은 생성·노드·엣지·컴파일·호출입니다

노드는 전체 상태가 아니라 바뀐 필드만 dict로 반환합니다

StateGraph

상태 스키마를 기준으로 그래프 빌더를 만듭니다.

START/END

실행의 시작점과 종료점을 명시합니다.

compile

연결 오류를 검증하고 실행 가능한 그래프로 바꿉니다.

01_introduction.ipynb

노트북 기준

```python
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    text: str
    word_count: int

def count_words(state: State) -> dict:
    return {"word_count": len(state["text"].split())}

builder = StateGraph(State)
builder.add_node("counter", count_words)
builder.add_edge(START, "counter")
builder.add_edge("counter", END)
graph = builder.compile()
print(graph.invoke({"text": "LangGraph 상태 그래프"}))
```

© 주식회사 똑똑한청년들 · 배움 에이아이

처음에는 그래프를 '제어 가능한 실행 로그'로 보는 편이 쉽습니다

그래프는 그림이 아니라 실행 중 상태가 어디서 바뀌었는지 추적하는 구조입니다

- **상태가 명시되면 테스트가 쉬워집니다**
  - 각 노드는 입력 상태와 출력 업데이트만 검증하면 됩니다
  - 조건부 엣지는 라우팅 함수 하나로 분리됩니다
- **컴파일된 그래프는 LangGraph 런타임 기능을 얻습니다**
  - stream, checkpointer, interrupt, get_state_history를 같은 방식으로 사용합니다
  - Studio와 서버 배포에서도 같은 그래프 객체를 대상으로 삼습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 03

Graph API

StateGraph의 기본 문법과 add_node 고급 옵션, Send·Command를 한 번에 정리합니다.

실습 · 03_langgraph/02_graph_api.ipynb

조건부 엣지는 상태 값을 다음 노드 이름으로 바꾸는 라우터입니다

라우팅 함수는 비즈니스 분류 결과를 그래프 제어 흐름으로 바꿉니다

classify

입력 질문을 category로 분류합니다.

route

category 값을 노드 이름으로 반환합니다.

add_conditional_edges

분기 가능한 목적지를 그래프에 등록합니다.

02_graph_api.ipynb — conditional edges

```python
from typing import Literal

def route(state: RouterState) -> Literal["weather", "math", "general"]:
    return state["category"]

builder = StateGraph(RouterState)
builder.add_node("classify", classify)
builder.add_node("weather", handle_weather)
builder.add_node("math", handle_math)
builder.add_node("general", handle_general)
builder.add_edge(START, "classify")
builder.add_conditional_edges("classify", route)
builder.add_edge("weather", END)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

입출력 스키마를 분리하면 내부 상태를 외부 응답에서 숨길 수 있습니다

프로덕션 API에서는 내부 중간값과 외부 응답 형식을 분리해야 합니다

InputSchema

호출자가 넣는 입력만 정의합니다.

InternalState

그래프 내부 계산에 필요한 중간 필드를 포함합니다.

OutputSchema

최종 사용자에게 돌려줄 필드만 노출합니다.

02_graph_api.ipynb — input/output schema

```python
class InputSchema(TypedDict):
    question: str

class OutputSchema(TypedDict):
    answer: str

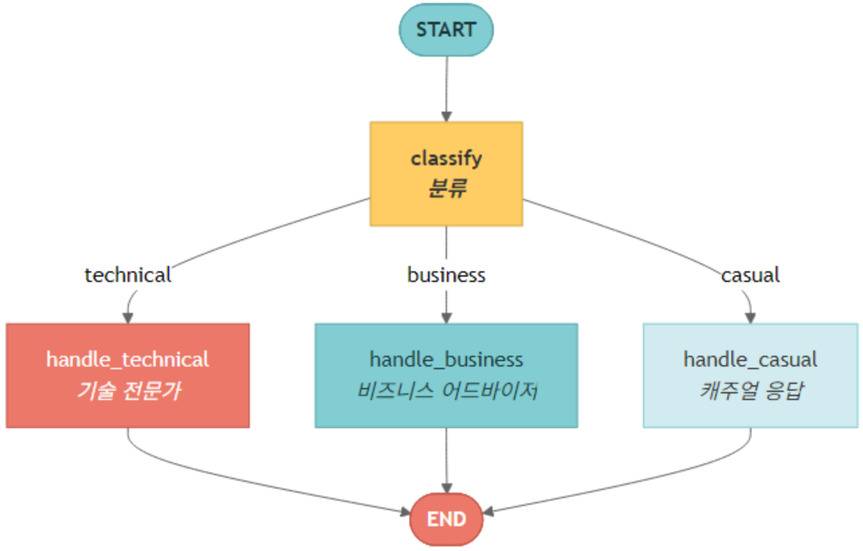
class InternalState(TypedDict):
    question: str
    answer: str
    intermediate: str

builder = StateGraph(
    InternalState,
    input_schema=InputSchema,
    output_schema=OutputSchema,
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

조건부 엣지는 그래프를 if문 더미가 아니라 라우팅 표면으로 만듭니다

분기 로직이 함수로 분리되면 테스트와 관측이 쉬워집니다



Conditional routing

분류 노드

state를 읽고 category 같은 라우팅 키를 만듭니다.

route 함수

다음 노드 이름을 Literal로 반환합니다.

핸들러 노드

각 경로는 독립적으로 테스트할 수 있습니다.

Annotated 리듀서는 상태를 덮어쓸지 누적할지 결정합니다

노드가 같은 키를 갱신해도 리듀서가 있으면 리스트처럼 안전하게 합쳐집니다

기본값

리듀서가 없으면 마지막 업데이트가 값을 덮어씁니다.

operator.add

리스트를 누적할 때 가장 자주 쓰는 패턴입니다.

메시지

MessagesState는 add_messages 리듀서를 미리 제공합니다.

02_graph_api.ipynb

Reducer

```python
from typing import Annotated, TypedDict
import operator

class AccState(TypedDict):
    items: Annotated[list[str], operator.add]
    count: int

def step_a(state: AccState) -> dict:
    return {"items": ["from_a"], "count": 1}

def step_b(state: AccState) -> dict:
    return {"items": ["from_b"], "count": 2}

builder = StateGraph(AccState)
builder.add_node("a", step_a)
builder.add_node("b", step_b)
builder.add_edge(START, "a")
builder.add_edge("a", "b")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Graph API의 운영 옵션은 노드 선언 시점에 붙입니다

retry, timeout, cache, error_handler, defer는 실행 정책이지 노드 내부 로직이 아닙니다

옵션	역할	적용 기준
retry_policy	일시 장애 자동 재시도	네트워크·일시적 API 오류
timeout	노드 실행 시간 제한	외부 호출이 긴 노드
cache_policy	같은 입력 결과 재사용	비싼 계산·검색
error_handler	실패 후 보정 경로	결제·파일 쓰기 같은 보정 필요 작업
defer	그래프 종료 직전 지연 실행	정리·알림·후처리

© 주식회사 똑똑한청년들 · 배움 에이아이

Send는 동적 fan-out을 만들고 Command는 상태 업데이트와 라우팅을 함께 표현합니다

map-reduce처럼 입력 개수에 따라 worker 수가 달라지는 흐름에서 Send가 핵심입니다

Send

같은 노드를 여러 입력으로 동적으로 호출합니다.

Command

update와 goto를 한 객체에 담아 라우팅합니다.

부모 그래프

Command.PARENT로 서브그래프 밖 라우팅도 표현합니다.

02_graph_api.ipynb

Send

```python
from typing import Annotated, TypedDict
import operator
from langgraph.types import Send

class FanOutState(TypedDict):
    items: list[str]
    results: Annotated[list[str], operator.add]

def dispatch(state: FanOutState) -> list[Send]:
    return [Send("worker", {"item": item}) for item in state["items"]]

def worker(state: dict) -> dict:
    return {"results": [f"처리: {state['item']}"]}

builder = StateGraph(FanOutState)
builder.add_node("worker", worker)
builder.add_conditional_edges(START, dispatch, ["worker"])
builder.add_edge("worker", END)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 04

Functional API

@entrypoint와 @task로 병렬 작업, previous 메모리, 캐싱, 구조화 interrupt를 다룹니다.

실습 · 03_langgraph/03_functional_api.ipynb

@task 결과는 future로 돌아오고 result()를 호출할 때 값이 필요해집니다

Functional API의 병렬성은 함수처럼 보이지만 즉시 값이 아니라 future를 다룬다는 점에서 출발합니다

@task

재시도와 체크포인트가 가능한 작업 단위입니다.

future

task 호출 결과는 나중에 result()로 수집합니다.

@entrypoint

전체 실행의 진입점과 체크포인터를 정의합니다.

03_functional_api.ipynb

@task + @entrypoint

```python
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def fetch_data(url: str) -> str:
    return f"{url}에서 가져온 데이터"

@task
def process_data(data: str) -> str:
    return f"처리 완료: {data.upper()}"

@entrypoint(checkpointer=InMemorySaver())
def data_pipeline(url: str) -> str:
    raw = fetch_data(url).result()
    return process_data(raw).result()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Functional API는 편하지만 결정론성 요구사항을 지켜야 합니다

재실행 가능한 그래프는 같은 입력과 체크포인트에서 같은 흐름으로 돌아와야 합니다

지켜야 할 것

- entrypoint 안에서 외부 부작용을 무작정 실행하지 않습니다
- 오래 걸리거나 실패할 수 있는 작업은 @task로 분리합니다
- 체크포인터를 쓸 때 thread_id를 명시합니다

헷갈리기 쉬운 것

- print나 임시 변수는 저장되지 않습니다
- entrypoint.final은 반환값과 저장값을 분리할 때 사용합니다
- previous는 같은 thread에서 이전 저장값을 읽을 때 사용합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

@task는 체크포인트되는 비동기 작업 단위입니다

task 호출은 Future를 반환하고 .result()로 결과를 수집합니다

@task

재시도·타임아웃·캐싱을 붙일 수 있는 작업 단위입니다.

@entrypoint

워크플로 진입점이며 checkpointer를 받습니다.

thread_id

같은 entrypoint라도 실행 이력을 구분합니다.

03_functional_api.ipynb

노트북 기준

```python
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def fetch_data(url: str) -> str:
    return f"{url}에서 가져온 데이터"

@task
def process_data(data: str) -> str:
    return f"처리 완료: {data.upper()}"

@entrypoint(checkpointer=InMemorySaver())
def data_pipeline(url: str) -> str:
    raw = fetch_data(url).result()
    return process_data(raw).result()

cfg = {"configurable": {"thread_id": "pipe-1"}}
print(data_pipeline.invoke("https://example.com/api", cfg))
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Functional API의 핵심은 병렬 시작과 결정론적 재실행입니다

비결정적 작업은 task 안으로 넣어야 checkpoint 재개 시 결과가 흔들리지 않습니다

기능	코드 표면	의미
병렬 실행	여러 task를 먼저 호출 후 result 수집	독립 작업을 동시에 시작
previous	entrypoint 인자 previous	같은 thread의 이전 저장값 접근
entrypoint.final	value와 save 분리	사용자 반환값과 다음 실행 저장값 분리
결정론성	시간·LLM·IO는 task 내부	재개 시 같은 작업을 중복 실행하지 않음

© 주식회사 똑똑한청년들 · 배움 에이아이

previous와 entrypoint.final은 사용자 반환값과 저장값을 분리합니다

보이는 답변은 문자열이어도 다음 실행에는 숫자 상태를 저장할 수 있습니다

previous

이전 실행의 save 값이 자동 주입됩니다.

value

이번 호출자가 받는 반환값입니다.

save

다음 호출의 previous로 저장되는 값입니다.

previous.py — 단기 메모리

```python
@entrypoint(checkpointer=InMemorySaver())
def counter(increment: int, *, previous: int = 0) -> int:
    return (previous or 0) + increment
```

final.py — 반환/저장 분리

```python
@entrypoint(checkpointer=InMemorySaver())
def smart_counter(n: int, *, previous: int = 0) -> entrypoint.final[str, int]:
    total = (previous or 0) + n
    return entrypoint.final(value=f"현재 합계: {total}", save=total)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Functional API에서도 interrupt는 Command(resume=...)으로 재개합니다

사람 검토가 들어가는 작업은 checkpointer가 있어야 중단 지점을 기억합니다

interrupt

사람에게 보여줄 질문과 검토 대상을 구조화합니다.

Command

resume 값으로 중단 지점에 답을 전달합니다.

재개

같은 thread_id를 써야 같은 실행으로 이어집니다.

03_functional_api.ipynb — interrupt

```python
from langgraph.types import Command, interrupt

@task
def review(draft: str) -> str:
    feedback = interrupt({"question": "승인하시겠어요?", "draft": draft})
    if feedback.get("approve"):
        return draft
    return f"{draft} (수정: {feedback.get('comment', '')})"

@entrypoint(checkpointer=InMemorySaver())
def approval_flow(topic: str) -> str:
    return review(f"{topic} 초안").result()

cfg = {"configurable": {"thread_id": "hitl-1"}}
approval_flow.invoke("LangGraph", cfg)
approval_flow.invoke(Command(resume={"approve": True}), cfg)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 05

워크플로 패턴

Prompt chaining, parallelization, routing, orchestrator-worker, evaluator-optimizer를 StateGraph로 구현합니다.

실습 · 03_langgraph/04_workflows.ipynb

Send는 실행 시점에 워커 수가 정해지는 fan-out 패턴에 사용합니다

고정된 분기보다 동적으로 여러 작업을 만들어야 할 때 Send가 필요합니다

dispatch

입력 리스트를 worker 호출 목록으로 바꿉니다.

worker

각 항목을 독립적으로 처리합니다.

reducer

Annotated reducer로 결과를 모읍니다.

04_workflows.ipynb — orchestrator-worker

```python
from langgraph.types import Send

def dispatch(state: FanOutState) -> list[Send]:
    return [Send("worker", {"item": it}) for it in state["items"]]

def worker(state: dict) -> dict:
    return {"results": [f"처리({state['item']})"]}

builder.add_conditional_edges(START, dispatch, ["worker"])
builder.add_edge("worker", "reduce")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

워크플로 패턴은 실패 지점과 비용 구조가 서로 다릅니다

패턴 이름보다 어떤 운영 문제가 생기는지 기준으로 선택해야 합니다

패턴	적합한 작업	운영 주의점
Prompt Chaining	검토 단계를 순서대로 통과하는 작업	앞 단계 오류가 뒤로 전파됩니다
Parallelization	서로 독립적인 분석 작업	결과 합성 기준이 필요합니다
Routing	질문 유형별 처리 분기	분류 오류가 곧 잘못된 경로가 됩니다
Orchestrator-Worker	작업 수가 입력마다 달라지는 경우	worker 결과를 줄이는 reducer가 중요합니다
Evaluator-Optimizer	생성·평가 반복 개선	반복 제한과 종료 기준이 필요합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

다섯 패턴은 'LLM을 몇 번 부르나'보다 제어 구조가 다릅니다

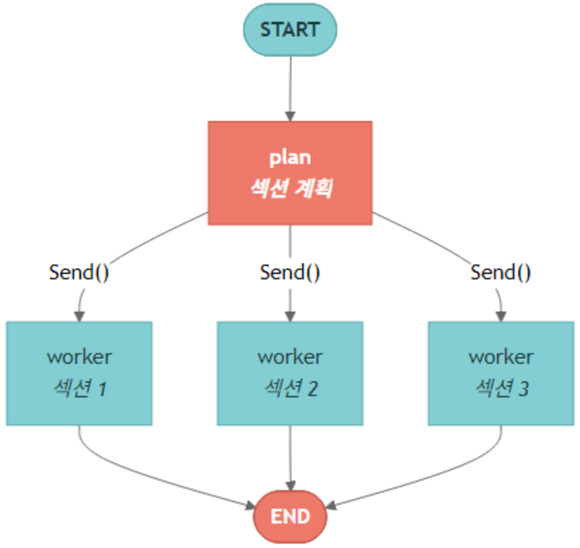
각 패턴은 실패 지점과 테스트 방식이 달라집니다

패턴	흐름	적합한 문제
Prompt Chaining	A 결과를 B 입력으로 전달	단계별 품질 게이트가 필요한 글쓰기
Parallelization	독립 작업 동시 실행 후 합치기	요약·분류·검토를 병렬로 처리
Routing	분류 결과로 다음 노드 선택	문의 유형별 전문 처리
Orchestrator-Worker	계획 후 worker 동적 생성	섹션 수가 입력마다 달라지는 보고서
Evaluator-Optimizer	생성·평가 반복	품질 기준을 통과할 때까지 개선

© 주식회사 똑똑한청년들 · 배움 에이아이

Orchestrator-Worker는 계획 결과만큼 worker를 동적으로 만듭니다

StateGraph에서는 Send()가 이 동적 fan-out의 연결점입니다



Orchestrator-Worker

Orchestrator

전체 작업을 보고 worker 입력 목록을 만듭니다.

Worker

각 작업 단위를 독립적으로 처리합니다.

Reducer

worker 결과를 리스트나 문서로 합칩니다.

Routing은 조건부 엣지 하나로 분류와 처리를 분리합니다

분류 노드를 테스트하면 라우팅 품질을 별도 지표로 볼 수 있습니다

classify

입력 질문을 category로 바꿉니다.

route

category를 다음 노드 이름으로 변환합니다.

handler

각 경로의 처리는 독립 함수가 맡습니다.

04_workflows.ipynb — Routing

```
from typing import Literal, TypedDict

class RouteState(TypedDict):
    query: str
    category: str
    result: str

def classify(state: RouteState) -> dict:
    q = state["query"].lower()
    if "계산" in q:
        return {"category": "math"}
    return {"category": "general"}

def route(state: RouteState) -> Literal["math", "general"]:
    return state["category"]

builder.add_conditional_edges("classify", route)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Evaluator-Optimizer는 품질 기준을 그래프의 반복 조건으로 만듭니다

평가 결과가 통과일 때만 END로 가고, 아니면 optimizer로 되돌립니다



GENERATE

모델이 첫 결과를 만듭니다.

EVALUATE

규칙이나 LLM으로 결과를 평가합니다.

ROUTE

기준 미달이면 개선 경로로 보냅니다.

OPTIMIZE

피드백을 반영해 다시 생성합니다.

이 패턴은 LLM을 믿는 대신 **검사 가능한 품질 루프**를 그래프에 넣는 방식입니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 06

ReAct 에이전트

Graph API와 Functional API로 도구 호출 루프를 직접 만들고 스트리밍과 메모리까지 연결합니다.

실습 · 03_langgraph/05_agents.ipynb

Graph API 에이전트는 모델 호출과 도구 실행을 명시적으로 나눕니다

ReAct를 그래프로 그리면 모델이 언제 도구를 부르고 언제 종료하는지 보입니다

구성	역할	코드 표면
call_model	메시지를 보고 다음 AIMessage를 생성합니다	model.bind_tools(tools)
ToolNode	AIMessage의 tool_calls를 실행합니다	ToolNode(tools)
tools_condition	도구 호출 여부에 따라 분기합니다	tools 또는 END
MessagesState	대화 메시지를 누적하는 상태입니다	messages reducer

© 주식회사 똑똑한청년들 · 배움 에이아이

Functional API 에이전트는 while 루프로 ReAct를 직접 표현합니다

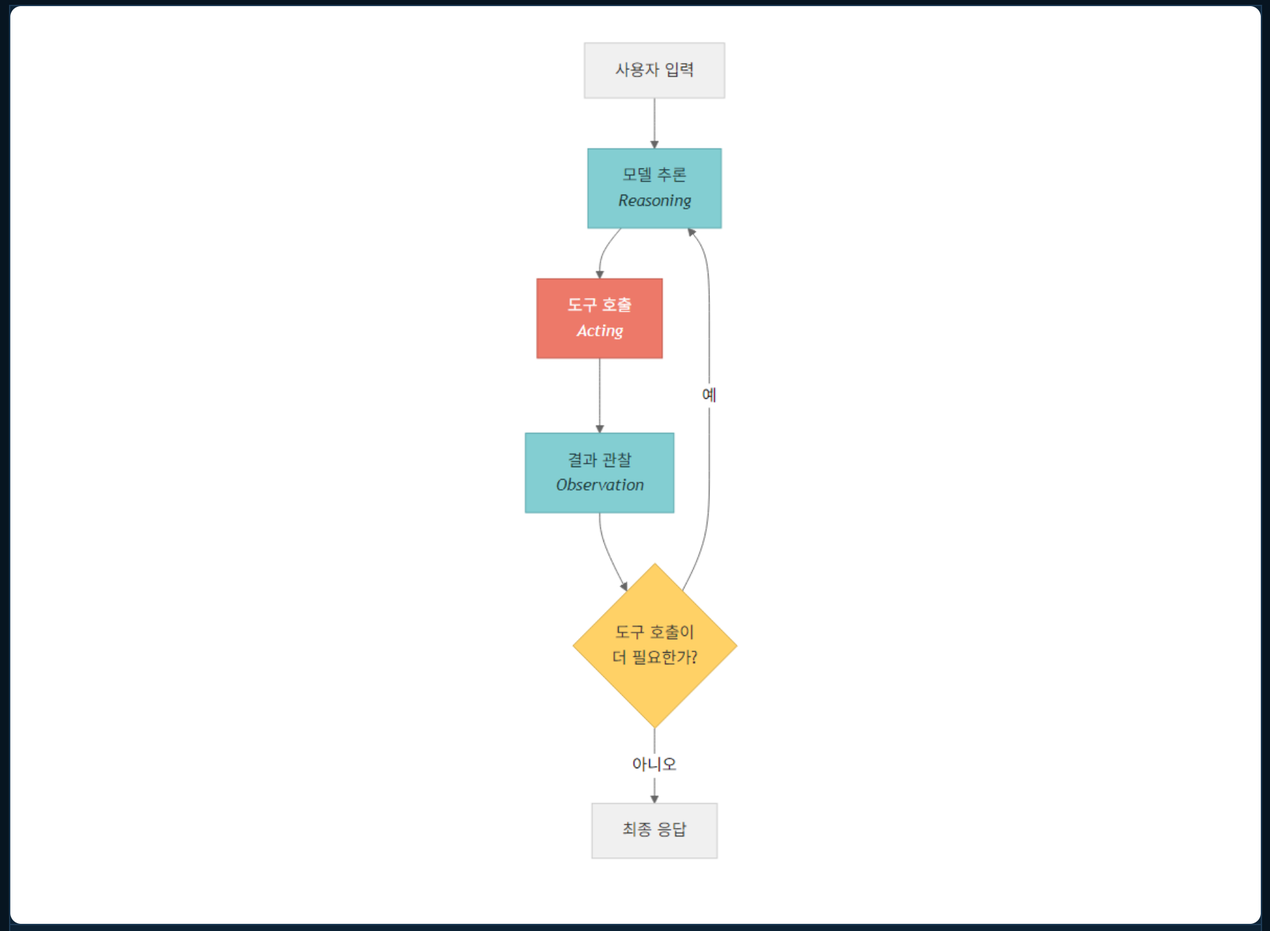
그래프 구조보다 코드 흐름을 읽는 것이 쉬운 팀에서는 Functional API 구현이 더 직관적일 수 있습니다

- **반복 루프**
  - 모델을 호출합니다
  - tool_calls가 없으면 최종 답변으로 종료합니다
  - tool_calls가 있으면 도구를 실행하고 메시지에 추가합니다
- **메모리 연결**
  - checkpointer가 있으면 같은 thread_id에서 대화가 이어집니다
  - thread_id가 다르면 대화 상태가 분리됩니다

© 주식회사 똑똑한청년들 · 배움 에이아이

ReAct 루프는 llm 노드와 tools 노드의 조건부 반복입니다

create_agent가 감추던 흐름을 StateGraph로 풀어 쓰면 디버깅 지점이 분명해집니다



ReAct loop

llm

도구 호출 여부를 판단하고 AIMessage를 생성합니다.

tools

tool_calls를 실행해 ToolMessage를 만듭니다.

should_continue

tool_calls가 있으면 tools, 없으면 END로 보냅니다.

bind_tools()는 모델이 호출할 수 있는 도구 스키마를 모델에 붙입니다

도구 함수 자체는 일반 Python 함수지만 모델에는 이름·설명·입력 스키마로 보입니다

@tool

함수명과 docstring을 도구 설명으로 바꿉니다.

tools_by_name

tool_call name으로 실제 함수를 찾기 위한 매핑입니다.

bind_tools

모델 호출 결과에 tool_calls가 생길 수 있게 합니다.

05_agents.ipynb

노트북 기준

```python
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

tools = [add, multiply]
tools_by_name = {tool.name: tool for tool in tools}
model_with_tools = model.bind_tools(tools)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

StateGraph ReAct 루프는 조건부 엣지 하나로 반복 여부를 결정합니다

마지막 AIMessage에 tool_calls가 있으면 도구 노드로, 없으면 종료로 갑니다

messages reducer

Annotated와 operator.add 또는 MessagesState로 메시지를 누적합니다.

ToolMessage

각 tool_call_id와 결과를 정확히 매칭해야 합니다.

checkpointer

메모리 있는 에이전트는 compile(checkpointer=...)로 만듭니다.

05_agents.ipynb — Graph API ReAct

```python
from typing import Literal
from langgraph.graph import END

def should_continue(state) -> Literal["tools", "__end__"]:
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END

builder.add_node("llm", llm_node)
builder.add_node("tools", tool_node)
builder.add_edge(START, "llm")
builder.add_conditional_edges("llm", should_continue, ["tools", END])
builder.add_edge("tools", "llm")
agent = builder.compile(checkpointer=InMemorySaver())
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Graph API와 Functional API 에이전트는 제어 표면이 다릅니다

같은 ReAct라도 어디에 복잡성이 드러나는지가 다릅니다

구분	Graph API	Functional API
흐름 표현	노드·엣지로 시각화	while 루프와 task 호출
디버깅	노드별 업데이트가 명확	함수 호출 흐름이 읽기 쉬움
복잡한 분기	조건부 엣지로 확장 용이	분기가 많으면 코드가 길어짐
간단한 에이전트	구조가 조금 장황	짧고 직관적

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 07

지속성과 메모리

checkpointer, get_state, get_state_history, update_state, InMemoryStore, trimming을 묶어 봅니다.

실습 · 03_langgraph/06_persistence_and_memory.ipynb

get_state와 get_state_history는 실행 상태를 운영자가 조회하는 표면입니다

체크포인터를 쓰면 현재 상태와 과거 실행 이력을 코드로 확인할 수 있습니다

get_state

현재 체크포인트 상태를 조회합니다.

get_state_history

노드별 실행 이력을 확인합니다.

config

thread_id가 조회 대상 실행을 정합니다.

06_persistence_and_memory.ipynb — state inspection

```
config = {"configurable": {"thread_id": "user-1"}}

current = graph.get_state(config)
print(current.values)

for snapshot in graph.get_state_history(config):
    print(snapshot.next, snapshot.values)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

update_state는 저장된 실행 상태를 외부에서 고치는 운영 도구입니다

오답 수정, 사람 검토, time travel은 저장 상태를 수정할 수 있다는 전제에서 작동합니다

as_node

어떤 노드가 수정한 것처럼 기록할지 정합니다.

values

덮어쓸 상태 변경분입니다.

thread_id

수정 대상 실행을 지정합니다.

06_persistence_and_memory.ipynb

update_state

```
config = {"configurable": {"thread_id": "user-1"}}

graph.update_state(
    config,
    {"messages": ["수정된 사용자 입력"]},
    as_node="human_review",
)

print(graph.get_state(config).values)
```

checkpointer를 붙이면 각 단계의 상태가 자동 저장됩니다

thread_id가 같으면 같은 실행 이력을 이어가고, 다르면 완전히 독립된 상태가 됩니다

InMemorySaver

실습용 체크포인터입니다. 프로세스 종료 시 사라집니다.

thread_id

대화 또는 실행 세션을 구분합니다.

get_state

현재 저장된 마지막 상태를 조회합니다.

06_persistence_and_memory.ipynb

노트북 기준

```python
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import MessagesState

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

graph = builder.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "chat-1"}}
graph.invoke({"messages": [{"role": "user", "content": "제 이름은 민수입니다."}]}, config)
print(graph.get_state(config).values)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

저장된 상태는 조회·이력 확인·수정까지 가능합니다

운영에서는 실패 분석과 수동 보정의 기본 도구가 됩니다

메서드	반환	사용 장면
get_state(config)	현재 체크포인트 상태	마지막 실행 결과 확인
get_state_history(config)	체크포인트 목록	어느 노드에서 상태가 바뀌었는지 추적
update_state(config, values)	새 체크포인트	사람이 상태를 고친 뒤 재개
invoke(None, config)	해당 지점부터 재개	Replay 또는 장애 복구

© 주식회사 똑똑한청년들 · 배움 에이아이

InMemoryStore는 thread를 넘어 공유되는 장기 메모리를 담당합니다

LangGraph 1.x 권장 패턴은 context_schema와 Runtime[Context]를 함께 쓰는 것입니다

store

사용자 선호 같은 장기 정보를 namespace별로 저장합니다.

context

user_id를 state가 아니라 호출 컨텍스트로 주입합니다.

Runtime

runtime.context와 runtime.store로 타입 안전하게 접근합니다.

06_persistence_and_memory.ipynb

Runtime + Store

```python
from dataclasses import dataclass
from langgraph.runtime import Runtime
from langgraph.store.memory import InMemoryStore

@dataclass
class Context:
    user_id: str

def remember(state: MessagesState, runtime: Runtime[Context]) -> dict:
    namespace = (runtime.context.user_id, "memories")
    runtime.store.put(namespace, "last", {"data": state["messages"][-1].content})
    return {}

store = InMemoryStore()
builder = StateGraph(MessagesState, context_schema=Context)
builder.add_node("remember", remember)
graph = builder.compile(checkpointer=InMemorySaver(), store=store)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

긴 대화는 저장만큼 삭제와 압축 전략이 중요합니다

메시지를 무한히 쌓으면 비용과 품질이 동시에 나빠집니다

- **trim_messages는 모델 호출 직전에 컨텍스트를 줄입니다**
  - system 메시지는 유지하고 최근 메시지를 우선 남기는 전략을 자주 씁니다
  - 토큰 예산을 기준으로 오래된 메시지를 잘라냅니다
- **RemoveMessage는 저장된 메시지를 명시적으로 제거합니다**
  - 실행 이력은 남기되 모델 입력에는 불필요한 과거를 줄일 수 있습니다
  - 요약 메모리와 함께 쓰면 장기 대화를 더 안정적으로 운영할 수 있습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 08

스트리밍

values, updates, messages, custom, v2 StreamPart, nostream, Event Streaming v3를 정리합니다.

실습 · 03_langgraph/07_streaming.ipynb

stream_mode는 누구에게 어떤 실행 정보를 보여줄지 정하는 선택입니다

사용자 UI와 운영 로그는 필요한 스트림 종류가 다릅니다

mode	보여주는 것	대표 용도
values	각 단계 이후 전체 상태	디버깅과 상태 스냅샷
updates	노드별 변경분	운영 로그와 진행 표시
messages	LLM 토큰 스트림	채팅 UI
custom	노드가 직접 내보낸 이벤트	도메인별 진행률
multiple	여러 모드를 함께 반환	UI와 로그 동시 처리

custom 스트리밍은 그래프 내부 진행률을 사용자 언어로 내보낼 때 사용합니다

토큰만 보여주면 장기 작업의 실제 진행 상태를 알기 어렵습니다

사용 시점

- 파일 처리, 검색, 분석처럼 중간 단계가 긴 작업입니다
- 사용자에게 현재 단계와 남은 작업을 설명해야 합니다

설계 기준

- 노드 이름 대신 사용자에게 의미 있는 이벤트명을 씁니다
- 성공 이벤트와 실패 이벤트를 구분합니다
- 프런트엔드에서 표시할 최소 필드를 정합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

stream_mode는 '무엇을 밖으로 보여줄지'의 선택입니다

최종 답만 필요한지, 노드 진행률이 필요한지, 토큰이 필요한지에 따라 모드가 달라집니다

모드	나오는 것	대표 사용
values	전체 상태 스냅샷	상태 전체를 디버깅
updates	노드별 변경분	실행 진행 로그
messages	LLM 토큰과 메타데이터	채팅 UI 토큰 출력
custom	writer가 보낸 임의 이벤트	진행률·중간 산출물 표시
multi-mode	여러 모드 튜플	UI와 로그를 동시에 구성

updates 스트림은 어느 노드가 어떤 상태를 바꿨는지 보여줍니다

복잡한 그래프에서는 최종값보다 노드별 변경분이 디버깅에 더 유용합니다

stream

invoke와 같은 입력을 받지만 iterator를 반환합니다.

updates

각 청크는 노드 이름과 변경된 필드를 담습니다.

UI

노드 이름을 단계 표시로 보여줄 수 있습니다.

07_streaming.ipynb

노트북 기준

```python
for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "LangGraph를 설명해 주세요."}]},
    stream_mode="updates",
):
    for node_name, update in chunk.items():
        print(f"[{node_name}] {list(update.keys())}")
```

version='v2'는 스트림 이벤트를 type·ns·data로 통일합니다

서브그래프와 메시지 스트림이 섞여도 같은 구조로 분기할 수 있습니다

- **StreamPart 구조**
  - chunk['type']. values, updates, messages, custom 같은 이벤트 종류입니다
  - chunk['ns']. 서브그래프 namespace가 있을 때 어느 그래프에서 온 이벤트인지 알려줍니다
  - chunk['data']. 실제 상태, 메시지, 커스텀 payload입니다
- **nostream 태그는 백그라운드 모델 토큰을 숨깁니다**
  - 내부 분류기나 요약기 토큰이 사용자 채팅 UI에 섞이지 않게 합니다
  - metadata의 tags와 chunk_position으로 특정 모델의 마지막 토큰을 감지할 수 있습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

앱 스트림은 projection을 정해 사용자에게 필요한 상태만 보냅니다

Event Streaming v3 관점에서는 그래프 내부 상태 전체가 아니라 화면 상태를 설계합니다



GRAPH

모든 노드 업데이트가 발생합니다.

PROJECT

사용자에게 보여줄 필드만 추립니다.

EVENT

토큰, 진행률, 결과 카드로 나눕니다.

RENDER

마지막 청크에서 커서를 정리합니다.

좋은 스트리밍은 많은 데이터를 보내는 것이 아니라 **사용자가 이해할 사건**만 보내는 것입니다.

PART 09

Interrupt와 Time Travel

사람 승인, 리뷰·편집, 도구 interrupt, 입력 검증, replay와 fork를 정리합니다.

실습 · 03_langgraph/08_interrupts_and_time_travel.ipynb

interrupt 패턴은 승인·수정·입력·검증·도구 승인으로 나눠 볼 수 있습니다

사람 개입은 중단 후 어떤 값을 받아 재개할지의 설계입니다. 새 API는 resume 값의 스키마도 함께 표현할 수 있습니다

패턴	resume 값	대표 상황
Approval	True 또는 False	삭제, 결제, 외부 전송 승인
Review & Edit	수정된 문자열 또는 객체	보고서 초안 검토
Input Collection	사용자 입력 값	누락 정보 보충
Validation	검증 결과와 수정 요청	정책 준수 확인
Tool Approval	도구 호출 허용/거절	위험 도구 실행 전 확인

© 주식회사 똑똑한청년들 · 배움 에이아이

Command(resume=...)은 중단된 그래프에 사람의 결정을 다시 넣습니다

interrupt가 값을 요구하면 Command로 값을 전달해 같은 실행을 이어갑니다

interrupt

노드 안에서 실행을 멈추고 값을 요청합니다.

Command

resume 값으로 중단 지점을 재개합니다.

checkpointer

중단 지점을 저장하기 위한 필수 전제입니다.

response_schema

LangGraph 1.2.12부터 interrupt(value, response_schema=...)로 재개 값의 형태를 명시할 수 있습니다.

08_interrupts_and_time_travel.ipynb — interrupt resume

```
from typing_extensions import TypedDict
from langgraph.types import Command, interrupt

class Approval(TypedDict):
    approved: bool
    reason: str

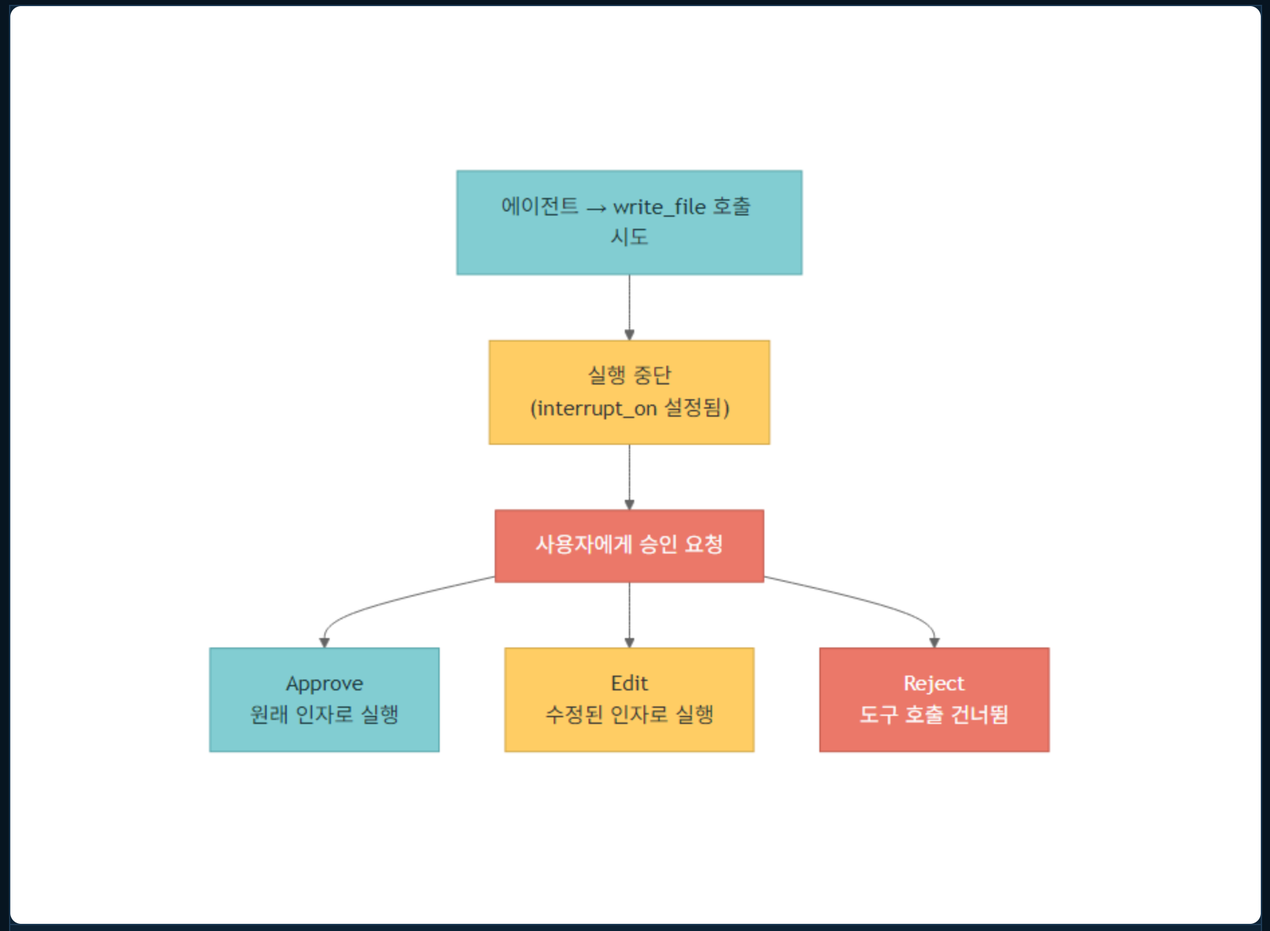
def approval_node(state):
    decision = interrupt(
        {"question": "계속 진행할까요?"},
        response_schema=Approval,
    )
    return {"approved": decision["approved"]}

result = graph.invoke(Command(resume={"approved": True, "reason": "ok"}),
config=config)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Human-in-the-loop는 그래프 실행을 승인 가능한 지점에서 멈춥니다

interrupt는 예외가 아니라 체크포인트와 함께 쓰는 정상 제어 흐름입니다



HITL flow

interrupt

사람에게 보여줄 값과 질문을 반환합니다.

checkpoint

중단 직전 상태가 저장되어 있어야 합니다.

Command(resume)

사람의 답을 같은 thread_id로 전달해 재개합니다.

표준 interrupt 패턴은 resume 값의 모양으로 구분됩니다

승인·편집·도구 승인·입력 검증·다중 interrupt를 모두 같은 재개 원리로 다룹니다

패턴	resume 값	사용 예
Approval	bool 또는 response_schema에 맞는 객체	메일 발송 전 승인
Review & Edit	수정된 문자열	초안 문장 편집
Tool Interrupt	action dict	도구 인자 일부 수정 후 승인
Input Validation	스키마 검증을 통과한 새 입력	나이·금액 같은 값 재입력
Multiple Interrupts	interrupt_id별 응답	동시에 여러 승인 처리

© 주식회사 똑똑한청년들 · 배움 에이아이

타임 트래블은 replay와 fork를 구분해야 합니다

replay는 과거 지점부터 재실행하고, fork는 과거 상태를 수정한 새 분기를 만듭니다

Replay

과거 checkpoint config로 invoke(None, config)를 호출합니다.

Fork

update_state로 값을 바꾸고 새 checkpoint를 만듭니다.

as_node

어느 노드가 상태를 쓴 것처럼 기록할지 지정합니다.

08_interrupts_and_time_travel.ipynb — Replay vs Fork

```
history = list(graph.get_state_history(config))
before_write = next(s for s in history if s.next == ("write_joke",))

# Replay: 저장된 topic은 재사용하고 write_joke만 다시 실행
replay = graph.invoke(None, before_write.config)

# Fork: 과거 상태를 고쳐 새 분기 생성
fork_config = graph.update_state(
    before_write.config,
    values={"topic": "dogs"},
    as_node="generate_topic",
)
forked = graph.invoke(None, fork_config)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

interrupt는 response_schema와 함께 UI 계약이 됩니다

완료 여부만 보는 대신, interrupt가 요구하는 입력 형태를 UI와 재개 코드가 공유하도록 설계합니다

- **완료되지 않았으면 out.interrupts에 질문이 들어 있습니다**
  - UI는 value와 response_schema를 승인 폼이나 편집 폼으로 렌더링합니다
  - 재개할 때는 같은 config와 Command(resume=...)을 사용합니다
- **완료되었으면 out.value가 최종 상태입니다**
  - v1 호환 dict 접근은 남아 있지만 v2 스타일로 옮기는 편이 명확합니다
  - 서브그래프 타임트래블도 v2에서 더 일관된 분기를 제공합니다

PART 10

서브그래프

상태 스키마가 다른 서브그래프 래핑, 직접 노드 추가, 서브그래프 스트리밍과 checkpointer 모드를 다룹니다.

실습 · 03_langgraph/09_subgraphs.ipynb

서브그래프 연결 방식은 상태 스키마가 같은지에 따라 달라집니다

큰 그래프를 나눌 때는 코드 재사용보다 상태 경계를 먼저 봐야 합니다

방식	조건	주의점
직접 추가	부모와 자식 상태 키가 맞습니다	공유 키의 reducer 충돌을 확인합니다
Wrapper node	상태 스키마가 다릅니다	입력 변환과 출력 매핑을 명시합니다
LLM subgraph	전문 흐름을 독립 그래프로 둡니다	프롬프트와 메시지 경계를 분리합니다
stream_subgraphs	부모·자식 실행을 함께 관찰합니다	UI 표시 레벨을 정해야 합니다

서브그래프는 파일 분리보다 실행 경계 설계가 중요합니다

서브그래프를 만들면 독립성이 생기지만 상태 변환과 디버깅 복잡도도 늘어납니다

- **분리하면 좋은 경우**
  - 팀이나 도메인이 다른 실행 흐름입니다
  - 하위 그래프만 따로 테스트하고 싶습니다
  - 반복 재사용되는 전문 워크플로입니다
- **분리하지 않는 편이 나은 경우**
  - 상태 키 몇 개만 추가하면 되는 단순 단계입니다
  - 부모와 자식이 거의 모든 상태를 공유합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

서브그래프는 그래프 안에 들어가는 독립 그래프입니다

작은 그래프를 따로 컴파일해 부모 그래프 노드처럼 사용할 수 있습니다

Call inside node 상태가 다를 때	부모 상태를 서브그래프 입력으로 변환하고 결과를 다시 부모 상태로 합칩니다.	`subgraph.invoke(...)`
Add as node 상태가 맞을 때	공유 키가 있는 그래프는 add_node(name, subgraph)로 직접 추가합니다.	`builder.add_node('sub', subgraph)`
Namespace 관측과 저장	서브그래프 스트림과 체크포인트는 namespace로 어느 내부 그래프인지 구분합니다.	`subgraphs=True`

© 주식회사 똑똑한청년들 · 배움 에이아이

상태 스키마가 다르면 래퍼 노드가 부모와 자식 상태를 변환합니다

서브그래프 내부 구조는 숨기되, 부모 그래프 입출력 계약은 명확히 둡니다

subgraph

분석 전용 그래프를 별도 컴파일합니다.

wrapper

부모 state를 서브그래프 입력으로 바꿉니다.

merge

서브그래프 결과만 부모 state에 되돌립니다.

09_subgraphs.ipynb

노트북 기준

```python
def run_analysis(state: PipelineState) -> dict:
    result = analysis_graph.invoke({"text": state["document"]})
    return {"analysis": result["summary"]}

parent_builder = StateGraph(PipelineState)
parent_builder.add_node("analysis", run_analysis)
parent_builder.add_edge(START, "analysis")
parent_builder.add_edge("analysis", END)
parent = parent_builder.compile()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

서브그래프 checkpointer는 세 가지 모드로 생각합니다

메모리와 interrupt 가능 여부가 모드마다 달라집니다

모드	설정	주의점
Per-Invocation	checkpointer 미지정	호출마다 fresh, 부모 checkpointer 상속
Per-Thread	checkpointer=True	thread별 누적 메모리, 병렬 tool 호출은 피함
Stateless	checkpointer=False	checkpoint가 없어 interrupt 불가
Namespace Isolation	고유 노드 이름으로 감싸기	여러 per-thread subagent 상태 충돌 방지

© 주식회사 똑똑한청년들 · 배움 에이아이

여러 per-thread 서브그래프는 고유 노드 이름으로 namespace를 격리합니다

agent 이름과 StateGraph 노드 이름을 맞추면 checkpoint prefix가 안정적으로 나뉩니다

name

create_agent와 add_node에 같은 고유 이름을 씁니다.

wrap

각 서브에이전트를 작은 StateGraph로 감쌉니다.

isolate

fruit_agent와 veggie_agent의 state가 섞이지 않습니다.

09_subgraphs.ipynb

Namespace

```python
from langgraph.graph import MessagesState, StateGraph
from langchain.agents import create_agent

def create_sub_agent(model: str, *, name: str, **kwargs):
    agent = create_agent(model=model, name=name, **kwargs)
    return (
        StateGraph(MessagesState)
        .add_node(name, agent)
        .add_edge("__start__", name)
        .compile()
    )

fruit_agent = create_sub_agent(
    "gpt-5.4-mini",
    name="fruit_agent",
    tools=[fruit_info],
    checkpointer=True,
)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 11

프로덕션 운영

langgraph.json, Studio, Agent Chat UI, GenericFakeChatModel, LangSmith tracing을 정리합니다.

실습 · 03_langgraph/10_production.ipynb

langgraph.json은 그래프를 서버와 Studio가 찾을 수 있게 하는 배포 계약입니다

노트북 코드가 앱이 되려면 그래프 경로, 의존성, 환경 변수가 명시돼야 합니다

graphs

그래프 이름과 Python 객체 경로를 연결합니다.

dependencies

서버가 설치할 패키지나 로컬 경로입니다.

env

실행에 필요한 환경 변수 파일입니다.

10_production.ipynb

langgraph.json

```
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./agent.py:graph"
  },
  "env": ".env"
}
```

프로덕션 테스트는 결정론 테스트와 LLM 테스트를 분리해야 합니다

그래프 구조가 맞는지와 모델 답변이 좋은지는 서로 다른 검증입니다

테스트	확인 대상	도구
결정론 테스트	상태 전이, 라우팅, reducer	일반 unit test
LLM 에이전트 테스트	도구 호출 순서와 최종 메시지	GenericFakeChatModel
회귀 테스트	이전 질문 세트의 품질 유지	dataset/eval
관측성	실패 지점과 비용 추적	trace/Studio

© 주식회사 똑똑한청년들 · 배움 에이아이

LangGraph 앱 구조는 그래프 객체를 서버와 Studio가 찾을 수 있게 만듭니다

코드에서 graph를 export하고 langgraph.json이 그 위치를 가리킵니다

항목	역할	확인 기준
langgraph.json	dependencies, graphs, env 선언	서버가 graph를 import할 수 있음
LangGraph Studio	로컬 그래프 시각 디버깅	상태와 노드 실행을 눈으로 확인
Agent Chat UI	채팅형 테스트 표면	streaming과 thread 상태를 빠르게 점검
테스트	결정론적 노드·fake model	LLM 호출 없이 핵심 경로 검증
관측성	LangSmith tracing	run, tag, metadata, anonymizer 확인

© 주식회사 똑똑한청년들 · 배움 에이아이

langgraph.json은 그래프 엔트리포인트를 선언하는 배포 계약입니다

로컬 개발 서버와 Studio는 이 파일을 기준으로 그래프를 로드합니다

dependencies

앱 실행에 필요한 패키지 또는 현재 프로젝트를 지정합니다.

graphs

그래프 이름과 Python import 경로를 매핑합니다.

env

환경변수 파일을 지정합니다.

10_production.ipynb

langgraph.json

```
import json

config = {
    "dependencies": ["."],
    "graphs": {"agent": "./agent.py:graph"},
    "env": ".env",
}
print(json.dumps(config, indent=2))
print("pip install 'langgraph-cli[inmem]'")
print("langgraph dev")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

GenericFakeChatModel은 LLM 테스트를 결정론적으로 만듭니다

모델 답을 고정하면 그래프 라우팅과 상태 업데이트를 자동 테스트할 수 있습니다

fake model

예상 메시지 iterator를 모델처럼 씁니다.

assert

결과 메시지와 state를 검증합니다.

비용 없음

실제 API 호출 없이 CI에서 반복 실행할 수 있습니다.

10_production.ipynb

테스트

```
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage, HumanMessage

fake_model = GenericFakeChatModel(
    messages=iter([AIMessage(content="The answer is 42.")])
)

def chatbot(state: MessagesState) → dict:
    return {"messages": [fake_model.invoke(state["messages"])]}

result = test_graph.invoke({"messages": [HumanMessage(content="테스트")]})
assert "42" in result["messages"][-1].content
```

관측성은 trace를 남기는 것보다 민감정보를 남기지 않는 설계가 먼저입니다

LangSmith tracing은 강력하지만 anonymizer와 metadata 정책을 함께 정해야 합니다

- **Selective tracing**
  - tracing_context로 특정 호출만 켜고 끌 수 있습니다
  - project, tag, metadata를 동적으로 부여해 실험과 운영을 분리합니다
- **Data privacy**
  - LangChainTracer와 Client anonymizer로 로그 직전 마스킹을 적용합니다
  - 사용자 입력 전체를 남길지, 추출 필드만 남길지 팀 정책을 먼저 정합니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 12

로컬 서버

LangGraph CLI, langgraph dev, Studio, Python SDK, REST API 호출 흐름을 정리합니다.

실습 · 03_langgraph/11_local_server.ipynb

로컬 서버는 같은 그래프를 Studio, SDK, REST에서 호출하게 합니다

강의용 노트북을 서비스 표면으로 바꾸는 중간 단계입니다

표면	용도	확인할 것
LangGraph CLI	개발 서버 실행	langgraph dev
Studio	그래프 시각 디버깅	상태와 노드 이동 확인
Python SDK	비동기 클라이언트 호출	threads와 runs
REST API	외부 앱 연동	stream 응답 형식

SDK 호출은 thread 생성과 run stream을 분리해서 이해해야 합니다

서버 모드에서는 그래프 실행이 로컬 함수 호출이 아니라 원격 run으로 관리됩니다

기본 순서

- client를 생성합니다
- thread를 만듭니다
- assistant 또는 graph id로 run을 시작합니다
- stream 이벤트를 순서대로 처리합니다

운영 관점

- thread는 대화와 실행 상태의 단위입니다
- run id는 개별 실행 추적 단위입니다

© 주식회사 똑똑한청년들 · 배움 에이아이

로컬 서버는 노트북 그래프를 서비스 호출 표면으로 바꿉니다

개발 중에는 in-memory 서버로 빠르게 돌리고, 배포 전에는 동일 API로 검증합니다

단계	명령·코드	확인
CLI 설치	pip install 'langgraph-cli[inmem]'	langgraph 명령 사용 가능
프로젝트 생성	agent.py + langgraph.json	그래프 import 경로 확인
개발 서버	langgraph dev	localhost:2024 실행
Studio	smith.langchain.com/studio	baseUrl로 로컬 서버 연결
SDK/REST	get_client, get_sync_client, curl	runs.stream 응답 확인

© 주식회사 똑똑한청년들 · 배움 에이아이

Python SDK는 서버 실행 중인 그래프를 비동기 스트림으로 호출합니다

노트북의 graph.invoke 감각을 서버 API로 옮기는 단계입니다

get_client

로컬 서버 URL을 지정합니다.

runs.stream

그래프 이름과 input을 넘기고 이벤트를 순회합니다.

event/data

프론트엔드와 백엔드는 같은 event payload를 기준으로 연결됩니다.

11_local_server.ipynb

SDK

```python
from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={"messages": [{"role": "human", "content": "What is LangGraph?"}]},
    ):
        print(chunk.event, chunk.data)

asyncio.run(main())
```

동기 SDK와 REST는 같은 실행을 다른 클라이언트 표면으로 부릅니다

서버를 붙인 뒤에는 SDK, REST, UI가 같은 graph_id와 thread를 공유합니다

sync client

스크립트와 테스트에서 간단히 사용합니다.

stream_mode

messages-tuple 같은 UI 친화 모드를 지정합니다.

REST

언어와 관계없이 배포된 그래프를 호출할 수 있습니다.

11_local_server.ipynb — Sync client

```python
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")
for chunk in client.runs.stream(
    None,
    "agent",
    input={"messages": [{"role": "human", "content": "What is LangGraph?"}]},
    stream_mode="messages-tuple",
):
    print(chunk.event, chunk.data)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

배포 준비는 서버가 켜지는지보다 호출 계약이 안정적인지를 봅니다

입력 스키마, graph 이름, env, thread 정책이 흔들리면 UI와 운영 자동화가 함께 깨집니다

배포 전 체크

- langgraph.json의 graph 경로가 실제 객체를 import하는지 확인합니다
- 필수 환경변수와 선택 환경변수를 분리합니다
- SDK와 REST 모두 같은 샘플 입력으로 성공하는지 확인합니다

운영 연결

- Studio는 디버깅용, SDK/REST는 앱 연동용으로 역할을 나눕니다
- thread_id와 user context 정책을 문서화해야 상태 격리가 유지됩니다

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 13

내구성 실행

@task, 체크포인터, fault tolerance, graceful shutdown, DeltaChannel을 정리합니다.

실습 · 03_langgraph/12_durable_execution.ipynb

내구성 실행은 실패 복구를 위해 side effect와 checkpoint를 분리합니다

긴 작업은 처음부터 재실행해도 안전한 코드 구조가 필요합니다

요구사항	의미	설계 방법
Idempotency	같은 작업을 다시 해도 결과가 망가지지 않습니다	외부 쓰기 작업은 고유 id로 보호
Checkpoint	어디까지 끝났는지 저장합니다	checkpointer와 thread_id 사용
Task boundary	재시도 가능한 단위를 분리합니다	@task로 외부 작업 분리
Resume	실패 지점 이후부터 이어갑니다	durability mode와 상태 확인

© 주식회사 똑똑한청년들 · 배움 에이아이

@task로 외부 작업을 감싸면 재시도와 재개 경계가 선명해집니다

entrypoint 안에 모든 작업을 몰아넣으면 실패 후 어디부터 다시 시작할지 불분명합니다

@task

실패하거나 오래 걸리는 외부 작업 단위입니다.

entrypoint

작업 순서를 조립하는 진입점입니다.

checkpointer

완료된 task 결과를 저장합니다.

12_durable_execution.ipynb

durable task

```python
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def call_external_api(item: str) -> str:
    return f"처리 완료: {item}"

@entrypoint(checkpointer=InMemorySaver())
def durable_flow(items: list[str]) -> list[str]:
    return [call_external_api(x).result() for x in items]
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Graph API와 Functional API는 재개 시작점이 다릅니다

어디서부터 다시 실행되는지 알아야 중복 결제나 중복 파일 쓰기를 피할 수 있습니다

표면	재개 시작점	주의
StateGraph	중단된 노드의 시작	노드 내부 side effect는 idempotent해야 함
Functional API	entrypoint에서 시작	@task 결과는 checkpoint에서 복원
@task	작업 결과 저장	비결정적 IO와 LLM 호출을 task 안에 둠
checkpointer	단계별 상태 저장	thread_id가 재개의 기준

© 주식회사 똑똑한청년들 · 배움 에이아이

Functional API에서는 LLM 호출과 외부 IO를 @task로 감싸야 재개가 안정적입니다

entrypoint가 다시 시작되어도 이미 끝난 task 결과는 저장값에서 복원됩니다

generate_draft

비싼 작업 또는 LLM 호출을 task로 둡니다.

checkpointer

entrypoint에 연결합니다.

thread_id

장애 후 같은 실행으로 재개합니다.

12_durable_execution.ipynb @task

```python
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def generate_draft(topic: str) -> str:
    return f"Draft about {topic}"

@task
def review_draft(draft: str) -> str:
    return f"Reviewed: {draft}"

@entrypoint(checkpointer=InMemorySaver())
def write_document(topic: str) -> str:
    draft = generate_draft(topic).result()
    return review_draft(draft).result()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

LangGraph 1.2 fault tolerance는 timeout, retry, error handler를 더 명시적으로 다룹니다

장기 작업은 실패를 숨기는 것이 아니라 보정 가능한 상태로 남기는 설계가 필요합니다

- **TimeoutPolicy**
  - idle_timeout으로 노드가 멈춰 있는 시간을 제한합니다
  - heartbeat를 쓰면 장기 작업 중에도 살아 있음을 런타임에 알릴 수 있습니다
- **Graceful shutdown**
  - RunControl로 drain을 요청하고 GraphDrained를 받아 checkpoint 기반 재개를 준비합니다
  - runtime.drain_requested를 보고 노드 내부에서 안전하게 중단할 수 있습니다
- **DeltaChannel**
  - append-heavy 메시지나 로그에서 checkpoint 저장량을 줄이는 beta 채널입니다

© 주식회사 똑똑한청년들 · 배움 에이아이

장애 복구 시나리오는 체크포인트 확인에서 시작합니다

실패 후 처음 할 일은 재시작이 아니라 어떤 노드까지 완료됐는지 확인하는 것입니다



FAIL

오류가 발생하고 checkpoint가 남습니다.

INSPECT

마지막 완료 상태와 next 노드를 확인합니다.

FIX

필요하면 사람이나 운영 도구가 상태를 보정합니다.

RESUME

같은 config로 다음 지점부터 이어갑니다.

내구성 실행은 자동 재시도만이 아니라 **상태를 보고 고친 뒤 재개**하는 운영 절차입니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

PART 14

API 선택과 Pregel

Graph API, Functional API, 두 API 결합, Pregel 채널과 슈퍼스텝 실행 모델을 정리합니다.

실습 · 03_langgraph/13_api_guide_and_pregel.ipynb

Graph API와 Functional API 선택은 팀이 읽을 실행 표현을 기준으로 합니다

둘 중 하나만 맞는 것이 아니라 운영자가 이해할 표현을 고르는 문제입니다

기준	Graph API	Functional API
흐름 가시성	노드와 엣지가 문서처럼 보입니다	코드 호출 흐름이 그대로 보입니다
복잡한 분기	조건부 엣지와 Send가 명시적입니다	일반 Python 제어문으로 표현합니다
팀 협업	다이어그램과 리뷰에 유리합니다	개발자가 빠르게 읽기 쉽습니다
운영 기능	체크포인트·스트리밍·interrupt 사용	동일하게 사용 가능
Tracing	add_node(trace_policy=...)로 노드별 추적 payload를 조정합니다	미들웨어 trace_policy와 같은 운영 기준으로 봅니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Pregel은 노드가 superstep 단위로 메시지를 주고받는 내부 실행 모델입니다

LangGraph를 깊게 디버깅할 때는 그래프가 한 번에 전부 실행되는 것이 아니라 단계별로 동기화된다고 이해해야 합니다

핵심 관점

- 각 superstep에서 활성 노드가 실행됩니다
- 노드는 채널에 값을 쓰고 다음 superstep에서 읽힙니다
- 더 이상 활성 노드가 없으면 실행이 끝납니다

학습 효과

- reducer와 channel의 의미가 선명해집니다
- 병렬 fan-out/fan-in의 실행 순서를 설명할 수 있습니다

© 주식회사 똑똑한청년들 · 배움 에이아이

Graph API와 Functional API 선택은 팀의 디버깅 표면을 고르는 문제입니다

기능 차이보다 복잡성이 어디에 보이는지가 선택 기준입니다

기준	Graph API	Functional API
복잡한 분기	노드와 엣지로 드러남	함수 내부 if/while로 숨어듦
간단한 파이프라인	장황할 수 있음	읽기 쉽고 짧음
시각화	Studio에서 구조가 잘 보임	task 단위 흐름 중심
팀 협업	노드 소유권 분리 쉬움	함수 단위로 빠르게 작성
내구성	compile(checkpointer=...)	@entrypoint(checkpointer=...)

© 주식회사 똑똑한청년들 · 배움 에이아이

두 API는 한 앱 안에서 역할을 나눠 함께 쓸 수 있습니다

복잡한 조정은 Graph API, 단순 데이터 처리 worker는 Functional API로 분리할 수 있습니다

Graph API

상위 coordinator와 라우팅을 담당합니다.

Functional API

반복 가능한 worker 파이프라인을 담당합니다.

계약

서로 넘기는 입력과 출력 스키마만 고정합니다.

13_api_guide_and_pregel.ipynb

API 결합

```python
@task
def summarize(section: str) -> str:
    return f"요약: {section}"

@entrypoint(checkpointer=InMemorySaver())
def summarize_worker(section: str) -> str:
    return summarize(section).result()

def worker_node(state: dict) -> dict:
    result = summarize_worker.invoke(state["section"], state["config"])
    return {"summary": result}

coordinator = StateGraph(CoordinatorState)
coordinator.add_node("worker", worker_node)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

Pregel은 노드가 채널을 읽고 쓰는 슈퍼스텝 실행 모델입니다

StateGraph를 쓰면 직접 Pregel을 다루지 않아도 내부 모델을 이해할 수 있습니다

개념	의미	예시
NodeBuilder	채널 구독과 쓰기 선언	subscribe_only('a').write_to('b')
Channel	값 전달과 누적 방식	EphemeralValue, LastValue, Topic
Superstep	같은 단계의 노드가 병렬 실행	업데이트는 다음 step에서 관찰
BinaryOperatorAggregate	리듀서 기반 누적	StateGraph의 Annotated 리듀서와 연결
DeltaChannel	delta checkpoint 최적화	메시지·로그 누적량 절감

© 주식회사 똑똑한청년들 · 배움 에이아이

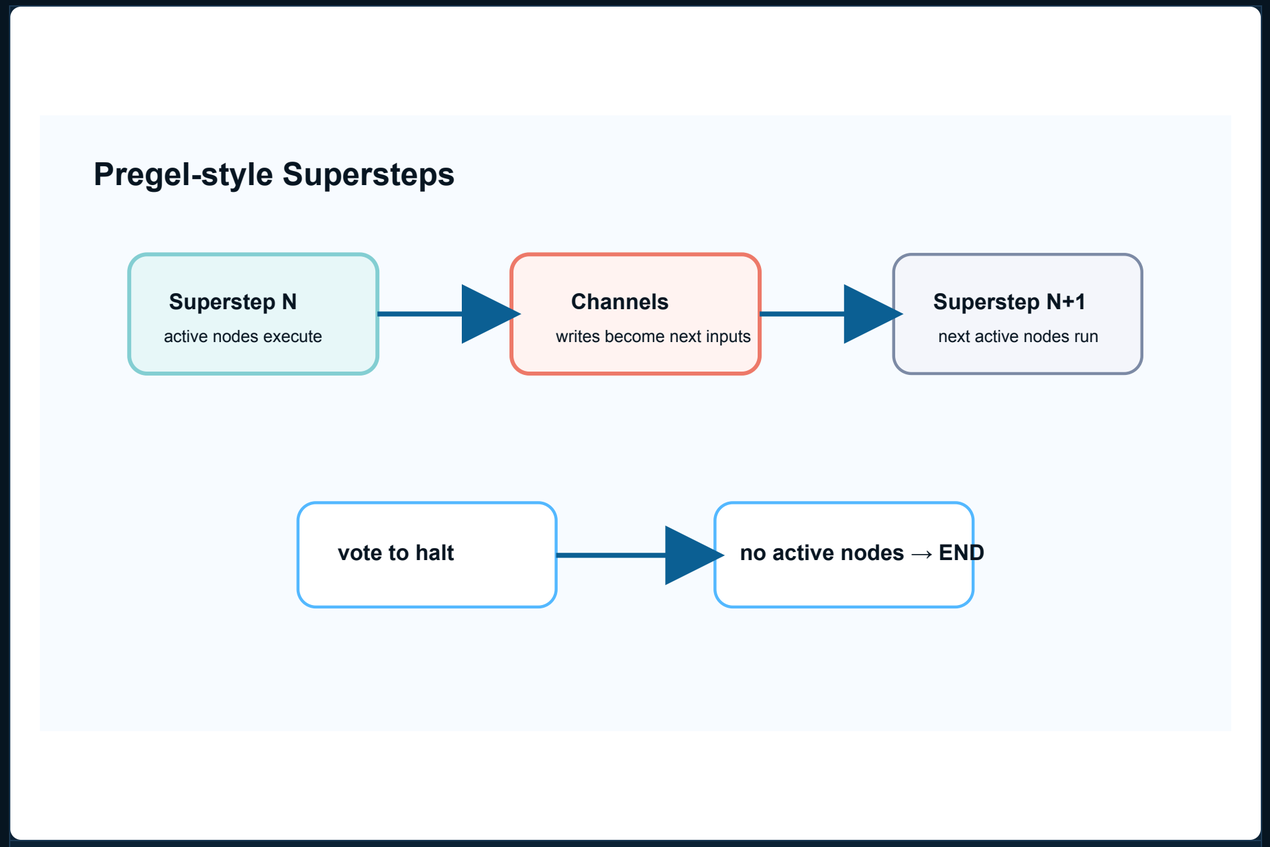
REFERENCE

개념 근거와 실행 모델

LangGraph 개념 설명은 Pregel 논문, LangGraph 공식 문서, ReAct 논문을 기준으로 보강합니다.

LangGraph의 superstep 관점은 Pregel의 반복 실행 모델로 설명할 수 있습니다

Pregel은 그래프 계산을 superstep의 반복으로 설명합니다. LangGraph의 Pregel API와 노드 실행 이해에 유용한 배경입니다.



Pregel-style superstep model

Superstep

활성 노드가 실행되고 값을 채널에 씁니다.

Message/Channel

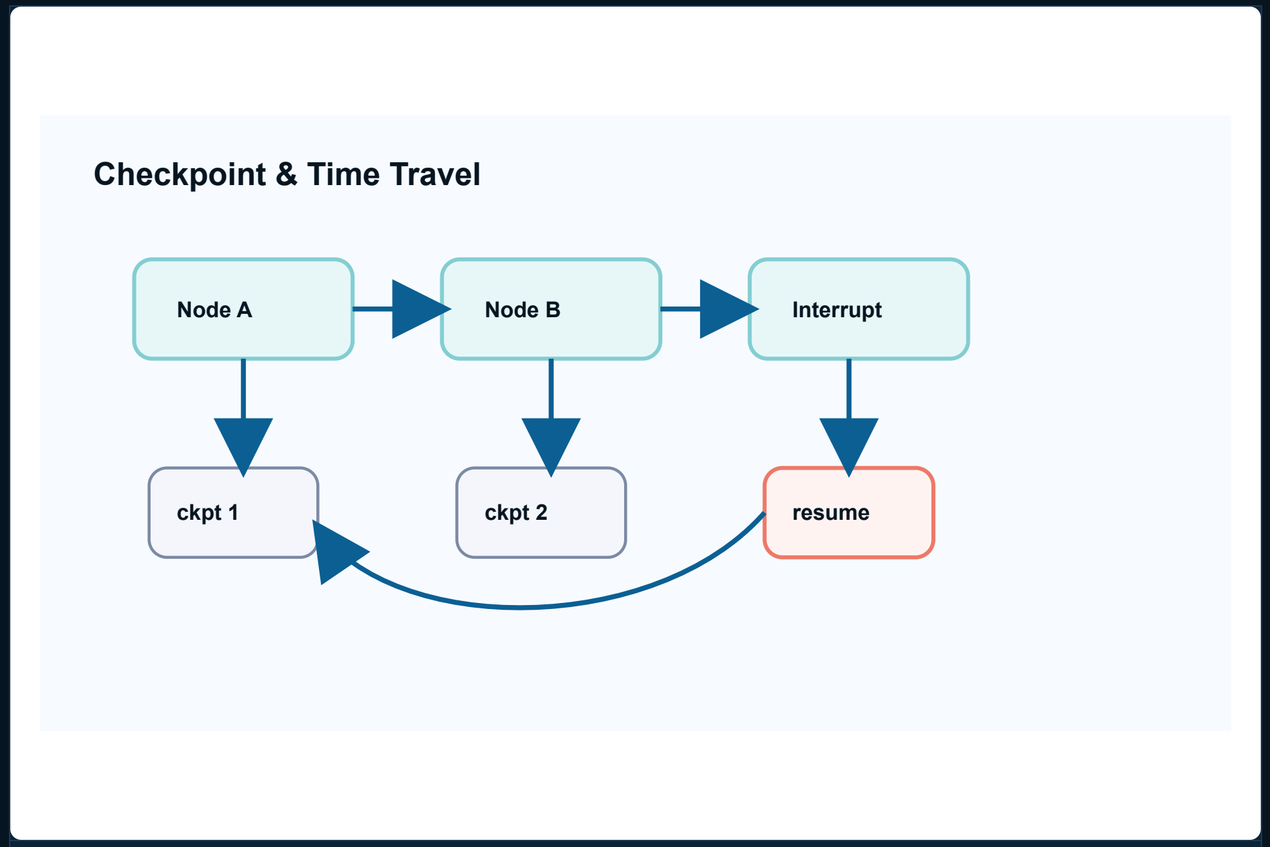
이번 단계의 출력은 다음 단계의 입력이 됩니다.

Halt

더 실행할 노드가 없으면 그래프가 종료됩니다.

Persistence와 time travel은 checkpoint를 기준으로 실행을 조회·수정·재개합니다

LangGraph 공식 문서는 checkpointer를 thread-scoped short-term memory, Store를 cross-thread long-term memory로 구분합니다.



checkpoint / time travel / resume

Checkpoint

각 실행 단계의 상태 스냅샷입니다.

Interrupt

사람 입력이 필요한 지점에서 실행을 멈춥니다.

Replay

이전 상태를 조회하거나 수정해 다른 경로를 실험합니다.

© 주식회사 똑똑한청년들 · 배움 에이아이

LangGraph 개념 보강 출처는 논문과 공식 문서를 기준으로 삼습니다

덱의 개념 설명은 노트북 코드와 아래 신뢰 출처를 함께 반영합니다.

개념	출처	덱 반영
ReAct loop	Yao et al., ReAct, arXiv:2210.03629	reasoning-action-observation 루프 설명
RAG	Lewis et al., RAG, NeurIPS 2020	검색 증강 생성과 provenance 설명
Pregel	Malewicz et al., SIGMOD 2010	superstep, message passing, halt 설명
LangGraph runtime	LangGraph official docs	durable execution, streaming, HITL, persistence 설명

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

01_introduction.ipynb

01_introduction.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/01_introduction.ipynb

01_introduction 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_introduction.ipynb

cell 7

```
import importlib

packages = {
    "langgraph": "langgraph",
    "langchain": "langchain",
    "langchain_openai": "langchain-openai",
}

print("=" * 50)
print("LangGraph 환경 확인")
print("=" * 50)

for module_name, package_name in packages.items():
    try:
        mod = importlib.import_module(module_name)
        version = getattr(mod, "__version__", "installed")
        print(f"  OK  {package_name}: {version}")
    except ImportError:
        print(f"  ERR {package_name}: 설치되지 않음")
```

01_introduction 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_introduction.ipynb

cell 8

```
from dotenv import load_dotenv
import os
load_dotenv(override=True)

required = ["OPENAI_API_KEY"]
optional = ["TAVILY_API_KEY", "LANGSMITH_API_KEY"]

print("API 키 상태:")
for key in required:
    print(f"  {'OK' if os.environ.get(key) else 'MISSING'} {key} (필수)")
for key in optional:
    print(f"  {'OK' if os.environ.get(key) else '--'} {key} (선택)")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_introduction 코드 03

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

01_introduction.ipynb

cell 9

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

01_introduction 코드 04

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

01_introduction.ipynb

cell 10

```python
# Core import verification
from langgraph.graph import StateGraph, START, END
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command, interrupt
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, AIMessage
from langchain_openai import ChatOpenAI

print("모든 핵심 임포트 완료")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

01_introduction 코드 05

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

01_introduction.ipynb

cell 12

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class SimpleState(TypedDict):
    input: str
    output: str

def greet(state: SimpleState) -> dict:
    return {"output": f"안녕하세요, {state['input']}님!"}

builder = StateGraph(SimpleState)
builder.add_node("greet", greet)
builder.add_edge(START, "greet")
builder.add_edge("greet", END)

graph = builder.compile()
result = graph.invoke({"input": "LangGraph"}, config=lf_config)
print("Graph API 결과:", result["output"])
```

01_introduction 코드 06

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

01_introduction.ipynb

cell 14

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver

@task
def process(text: str) → str:
    return f"처리 완료: {text.upper()}"

@entrypoint(checkpointer=InMemorySaver())
def simple_workflow(text: str) → str:
    result = process(text).result()
    return result

output = simple_workflow.invoke("안녕하세요 langgraph", {**{"configurable": {"thread_id": "demo"}}, **lf_config})
print("Functional API 결과:", output)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

02_graph_api.ipynb

02_graph_api.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/02_graph_api.ipynb

02_graph_api 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_graph_api.ipynb

cell 4

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

02_graph_api.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

02_graph_api 코드 03 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 7

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict


class State(TypedDict):
    text: str
    word_count: int


def count_words(state: State) -> dict:
    words = state["text"].split()
    return {
        "word_count": len(words)
    }


builder = StateGraph(State)

builder.add_node("counter", count_words)

builder.add_edge(START, "counter")
builder.add_edge("counter", END)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 03 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 7

```
graph = builder.compile()


result = graph.invoke(
    {
        "text": "LangGraph는 에이전트를 구축하기 위한 강력한 프레임워크입니다."
    },
    config=lf_config
)

print(f"텍스트: {result['text']}")
print(f"단어 수: {result['word_count']}")
```

02_graph_api 코드 04 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 9

```
from typing import Annotated, TypedDict
import operator

class AccState(TypedDict):
    items: Annotated[list[str], operator.add]  # list append via reducer
    count: int  # default: override


def step_a(state: AccState) -> dict:
    return {
        "items": ["from_a"],
        "count": 1
    }


def step_b(state: AccState) -> dict:
    return {
        "items": ["from_b"],
        "count": 2
    }
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 04 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 9

```
builder = StateGraph(AccState)

builder.add_node("a", step_a)
builder.add_node("b", step_b)

builder.add_edge(START, "a")
builder.add_edge("a", "b")
builder.add_edge("b", END)

graph = builder.compile()

result = graph.invoke(
    {
        "items": [],
        "count": 0
    },
    config=lf_config
)

print(f"items (리듀서=add): {result['items']}")
# ["from_a", "from_b"]
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 04 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 9

```
print(f"count (덮어쓰기): {result['count']}")
# 2
```

02_graph_api 코드 05 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 11

```
from typing import Literal

class RouterState(TypedDict):
    query: str
    category: str
    result: str

def classify(state: RouterState) -> dict:
    q = state["query"].lower()
    if "weather" in q or "날씨" in q:
        return {"category": "weather"}
    elif "math" in q or "calculate" in q or "계산" in q:
        return {"category": "math"}
    return {"category": "general"}

def handle_weather(state: RouterState) -> dict:
    return {"result": f"[날씨] 날씨 확인 중: {state['query']}"}

def handle_math(state: RouterState) -> dict:
    return {"result": f"[수학] 계산 중: {state['query']}"}

def handle_general(state: RouterState) -> dict:
```

02_graph_api 코드 05 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 11

```
    return {"result": f"[일반] 처리 중: {state['query']}"}

def route(state: RouterState) -> Literal["weather", "math", "general"]:
    return state["category"]

builder = StateGraph(RouterState)
builder.add_node("classify", classify)
builder.add_node("weather", handle_weather)
builder.add_node("math", handle_math)
builder.add_node("general", handle_general)

builder.add_edge(START, "classify")
builder.add_conditional_edges("classify", route)
builder.add_edge("weather", END)
builder.add_edge("math", END)
builder.add_edge("general", END)

graph = builder.compile()

for query in ["오늘 날씨는?", "2+2를 계산해줘", "농담 하나 해줘"]:
    result = graph.invoke({"query": query}, config=lf_config)
    print(f"  {query} → {result['result']}")
```

02_graph_api 코드 06

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 13

```
from langgraph.graph import MessagesState
from langchain.messages import HumanMessage, SystemMessage

def chatbot(state: MessagesState) -> dict:
    response = model.invoke(state["messages"], config=lf_config)
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

graph = builder.compile()

result = graph.invoke({"messages": [HumanMessage(content="2+2는 얼마인가요?")]}, config=lf_config)
print("응답:", result["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 07 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 15

```python
from typing import TypedDict

class InputSchema(TypedDict):
    question: str


class OutputSchema(TypedDict):
    answer: str


class InternalState(TypedDict):
    question: str
    answer: str
    intermediate: str  # internal only


def process(state: InternalState) -> dict:
    return {
        "intermediate": state["question"].upper()
    }
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 07 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 15

```
def answer(state: InternalState) -> dict:
    return {
        "answer": f"'{state['intermediate']}'에 대한 답: 42"
    }


builder = StateGraph(
    InternalState,
    input_schema=InputSchema,
    output_schema=OutputSchema
)

builder.add_node("process", process)
builder.add_node("answer", answer)

builder.add_edge(START, "process")
builder.add_edge("process", "answer")
builder.add_edge("answer", END)

graph = builder.compile()

result = graph.invoke(
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 07 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 15

```
    {
        "question": "삶의 의미"
    },
    config=lf_config
)

print("출력:", result)

# intermediate 필드는 출력에 포함되지 않음
```

02_graph_api 코드 08 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 17

```python
from dataclasses import dataclass
from langgraph.runtime import Runtime


@dataclass
class ContextSchema:
    user_id: str
    llm_provider: str = "openai"


class CtxState(TypedDict):
    query: str
    answer: str


def ctx_node(state: CtxState, runtime: Runtime[ContextSchema]) -> dict:
    # State와 분리된 호출 시점 의존성
    ctx = runtime.context
    # 실행 메타데이터 (thread/run/attempt 등)
    info = runtime.execution_info
    return {
        "answer": (
```

02_graph_api 코드 08 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 17

```
            f"user={ctx.user_id}, provider={ctx.llm_provider}, "
            f"attempt={info.attempt} → '{state['query']}'"
        )
    }


builder = StateGraph(CtxState, context_schema=ContextSchema)
builder.add_node("ctx_node", ctx_node)
builder.add_edge(START, "ctx_node")
builder.add_edge("ctx_node", END)

graph = builder.compile()

result = graph.invoke(
    {"query": "hello"},
    context={"user_id": "alice", "llm_provider": "anthropic"},
    config=lf_config,
)
print(result["answer"])
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 09 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 19

```
from langgraph.types import RetryPolicy, CachePolicy
from langgraph.cache.memory import InMemoryCache


class ReliableState(TypedDict):
    payload: str
    out: str


# 첫 호출은 실패하고 두 번째에 성공 — retry_policy 데모용
_calls = {"n": 0}


def flaky(state: ReliableState) -> dict:
    _calls["n"] += 1
    if _calls["n"] < 2:
        raise ValueError("일시 장애")
    return {"out": f"처리: {state['payload']} (시도 {_calls['n']})"}


def heavy(state: ReliableState) -> dict:
    # 같은 입력이면 cache_policy(ttl=120)로 결과 재사용
```

02_graph_api 코드 09 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 19

```
    return {"out": state["out"].upper()}


builder = StateGraph(ReliableState)
builder.add_node(
    "flaky",
    flaky,
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError),
)
builder.add_node(
    "heavy",
    heavy,
    cache_policy=CachePolicy(ttl=120),
)
builder.add_edge(START, "flaky")
builder.add_edge("flaky", "heavy")
builder.add_edge("heavy", END)

graph = builder.compile(cache=InMemoryCache())

result = graph.invoke({"payload": "hello", "out": ""}, config=lf_config)
print(result["out"])
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 09 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 19

```
print(f"flaky 호출 횟수: {_calls['n']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

02_graph_api 코드 10 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 21

```
from langgraph.types import Send, Command


class FanOutState(TypedDict):
    items: list[str]
    results: Annotated[list[str], operator.add]


def dispatch(state: FanOutState) -> list[Send]:
    # map: 각 item마다 worker 인스턴스를 동적으로 생성
    return [Send("worker", {"item": it}) for it in state["items"]]


def worker(state: dict) -> dict:
    return {"results": [f"처리({state['item']})"]}


def reduce(state: FanOutState) -> dict:
    return {"results": [f"총 {len(state['results'])}건 집계"]}


builder = StateGraph(FanOutState)
```

02_graph_api 코드 10 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

02_graph_api.ipynb

cell 21

```
builder.add_node("worker", worker)
builder.add_node("reduce", reduce)
builder.add_conditional_edges(START, dispatch, ["worker"])
builder.add_edge("worker", "reduce")
builder.add_edge("reduce", END)

graph = builder.compile()

result = graph.invoke(
    {"items": ["a", "b", "c"], "results": []},
    config=lf_config,
)
for r in result["results"]:
    print(r)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

03_functional_api.ipynb

03_functional_api.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/03_functional_api.ipynb

03_functional_api 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_functional_api.ipynb cell 4

```python
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

03_functional_api.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

03_functional_api 코드 03 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb cell 7

```python
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


@task
def fetch_data(url: str) -> str:
    """URL에서 데이터를 가져오는 시뮬레이션입니다."""
    return f"{url}에서 가져온 데이터"


@task
def process_data(data: str) -> str:
    """가져온 데이터를 처리합니다."""
    return f"처리 완료: {data.upper()}"


@entrypoint(checkpointer=InMemorySaver())
def data_pipeline(url: str) -> str:
    raw = fetch_data(url).result()
    processed = process_data(raw).result()
    return processed
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 03 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 7

```
config = {
    "configurable": {
        "thread_id": "pipe-1"
    }
}

result = data_pipeline.invoke(
    "https://example.com/api",
    {**config, **lf_config}
)

print("파이프라인 결과:", result)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 04 (1/3)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 9

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


@task
def analyze_sentiment(text: str) -> str:
    return f"감성({text[:20]}...): 긍정적"


@task
def extract_keywords(text: str) -> str:
    words = text.split()
    return f"키워드: {', '.join(words[:3])}"


@task
def summarize(text: str) -> str:
    return f"요약: {text[:50]}..."


@entrypoint(checkpointer=InMemorySaver())
def parallel_analysis(text: str) -> dict:
```

03_functional_api 코드 04 (2/3)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 9

```
    # 3개 태스크를 병렬로 시작
    sentiment_future = analyze_sentiment(text)
    keywords_future = extract_keywords(text)
    summary_future = summarize(text)

    # 결과를 순서대로 수집
    return {
        "sentiment": sentiment_future.result(),
        "keywords": keywords_future.result(),
        "summary": summary_future.result(),
    }


config = {
    "configurable": {
        "thread_id": "parallel-1"
    }
}

result = parallel_analysis.invoke(
    "LangGraph는 상태 기반 에이전트를 구축하기 위한 강력한 프레임워크입니다",
    {**config, **lf_config}
```

03_functional_api 코드 04 (3/3)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb cell 9

```
)

for k, v in result.items():
    print(f"  {k}: {v}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 05

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 11

```python
@entrypoint(checkpointer=InMemorySaver())
def counter(increment: int, *, previous: int = 0) -> int:
    """호출할 때마다 이전 합계에 더합니다."""
    new_total = (previous or 0) + increment
    return new_total

config = {"configurable": {"thread_id": "counter-1"}}
print("호출 1:", counter.invoke(5, {**config, **lf_config}))   # 5
print("호출 2:", counter.invoke(3, {**config, **lf_config}))   # 8
print("호출 3:", counter.invoke(10, {**config, **lf_config}))  # 18
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 06

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 13

```python
@entrypoint(checkpointer=InMemorySaver())
def smart_counter(number: int, *, previous: int = 0) -> entrypoint.final[str, int]:
    """포맷된 문자열을 반환하되, 다음 호출을 위해 원시 숫자를 저장합니다."""
    new_total = (previous or 0) + number
    # value: 사용자에게 반환되는 값
    # save: 다음 호출의 previous로 저장되는 값
    return entrypoint.final(value=f"현재 합계: {new_total}", save=new_total)

config = {"configurable": {"thread_id": "smart-1"}}
print(smart_counter.invoke(10, {**config, **lf_config}))   # "현재 합계: 10"
print(smart_counter.invoke(5, {**config, **lf_config}))    # "현재 합계: 15"
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 07 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 15

```python
import time

# CORRECT: 비결정적 작업을 @task로 감싸기
@task
def get_timestamp() -> float:
    return time.time()


@task
def call_llm(prompt: str) -> str:
    response = model.invoke(prompt, config=lf_config)
    return response.content


@entrypoint(checkpointer=InMemorySaver())
def deterministic_workflow(prompt: str) -> dict:
    ts = get_timestamp().result()  # 체크포인트에 저장, 재실행 시 동일 값
    answer = call_llm(prompt).result()

    return {
        "timestamp": ts,
        "answer": answer
```

03_functional_api 코드 07 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 15

```
    }


config = {
    "configurable": {
        "thread_id": "det-1"
    }
}

result = deterministic_workflow.invoke(
    "한 단어로 인사해 주세요",
    {**config, **lf_config}
)

print(f"타임스탬프: {result['timestamp']}")
print(f"답변: {result['answer']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 08 (1/3)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb cell 17

```
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage
from langgraph.graph import add_messages

@tool
def add(a: int, b: int) → int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) → int:
    """두 수를 곱합니다."""
    return a * b

tools = [add, multiply]
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)

@task
def call_model(messages: list) → object:
    return model_with_tools.invoke(
        [SystemMessage(content="당신은 수학 도우미입니다.")] + messages,
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 08 (2/3)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 17

```
        config=lf_config,
)

@task
def call_tool(tool_call: dict) -> ToolMessage:
    tool_fn = tools_by_name[tool_call["name"]]
    result = tool_fn.invoke(tool_call["args"], config=lf_config)
    return ToolMessage(content=str(result), tool_call_id=tool_call["id"])

@entrypoint(checkpointer=InMemorySaver())
def agent(messages: list) -> list:
    llm_response = call_model(messages).result()

    while llm_response.tool_calls:
        tool_results = [call_tool(tc).result() for tc in llm_response.tool_calls]
        messages = add_messages(messages, [llm_response] + tool_results)
        llm_response = call_model(messages).result()

    return add_messages(messages, llm_response)

config = {"configurable": {"thread_id": "agent-1"}}
result = agent.invoke([HumanMessage(content="7 * 8 + 3은 얼마인가요?")], {**config, **lf_config})
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 08 (3/3)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb cell 17

```
print("에이전트 결과:", result[-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 09

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 19

```
from langgraph.types import RetryPolicy

# ValueError 만 재시도, 다른 예외는 즉시 전파
_attempts = {"n": 0}


@task(retry_policy=RetryPolicy(max_attempts=3, retry_on=ValueError))
def fetch_with_retry(url: str) -> str:
    _attempts["n"] += 1
    if _attempts["n"] < 2:
        raise ValueError(f"일시 장애 (시도 {_attempts['n']})")
    return f"{url} 응답 OK"


@entrypoint(checkpointer=InMemorySaver())
def reliable_pipeline(url: str) -> str:
    return fetch_with_retry(url).result()


cfg = {"configurable": {"thread_id": "retry-1"}}
print(reliable_pipeline.invoke("https://api.example.com", {**cfg, **lf_config}))
print(f"총 시도: {_attempts['n']}")
```

03_functional_api 코드 10 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 21

```
import time
from langgraph.cache.memory import InMemoryCache
from langgraph.types import CachePolicy


@task(cache_policy=CachePolicy(ttl=120))
def expensive_compute(x: int) -> int:
    # 1초 sleep → 같은 x 값으로 두 번째 호출은 캐시 히트로 즉시 반환
    time.sleep(1)
    return x * 2


@entrypoint(checkpointer=InMemorySaver(), cache=InMemoryCache())
def cached_pipeline(x: int) -> dict:
    t0 = time.perf_counter()
    a = expensive_compute(x).result()
    t1 = time.perf_counter()
    b = expensive_compute(x).result()  # 같은 입력 → 캐시 히트
    t2 = time.perf_counter()
    return {"a": a, "b": b, "first_s": round(t1 - t0, 2), "second_s": round(t2 - t1, 2)}
```

03_functional_api 코드 10 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 21

```
cfg = {"configurable": {"thread_id": "cache-1"}}
print(cached_pipeline.invoke(7, {**cfg, **lf_config}))
```

© 주식회사 똑똑한청년들 · 배움 에이아이

03_functional_api 코드 11 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 23

```
from langgraph.types import Command, interrupt


@task
def review(draft: str) -> str:
    # 구조화된 interrupt — 사람이 무엇을 검토하고 어떤 형식으로 응답하는지 명시
    feedback = interrupt({
        "question": "이 초안을 승인하시겠어요?",
        "draft": draft,
        "expected": "{'approve': bool, 'comment': str}",
    })
    if feedback.get("approve"):
        return draft
    return f"{draft} (수정: {feedback.get('comment', '')})"


@entrypoint(checkpointer=InMemorySaver())
def approval_flow(topic: str) -> str:
    draft = f"'{topic}'에 대한 한 문장 초안"
    return review(draft).result()
```

03_functional_api 코드 11 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

03_functional_api.ipynb

cell 23

```
cfg = {"configurable": {"thread_id": "hitl-1"}}

# 1차 호출 — interrupt 에서 멈춤
state = approval_flow.invoke("LangGraph 입문", {**cfg, **lf_config})
print("중단 신호:", state)

# 사람이 검토 후 재개
final = approval_flow.invoke(
    Command(resume={"approve": False, "comment": "출처 추가 요망"}),
    {**cfg, **lf_config},
)
print("최종:", final)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

04_workflows.ipynb

04_workflows.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/04_workflows.ipynb

04_workflows 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

04_workflows.ipynb cell 4

```
from dotenv import load_dotenv
load_dotenv(override=True)

from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

04_workflows.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

04_workflows 코드 03 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 7

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

class ChainState(TypedDict):
    topic: str
    draft: str
    improved: str

def generate_draft(state: ChainState) -> dict:
    response = model.invoke(f"다음에 대한 한 문장 사실을 작성해주세요: {state['topic']}", config=lf_config)
    return {"draft": response.content}

def improve_draft(state: ChainState) -> dict:
    response = model.invoke(f"다음 텍스트를 더 매력적으로 개선해주세요: {state['draft']}", config=lf_config)
    return {"improved": response.content}

builder = StateGraph(ChainState)
builder.add_node("draft", generate_draft)
builder.add_node("improve", improve_draft)
builder.add_edge(START, "draft")
builder.add_edge("draft", "improve")
builder.add_edge("improve", END)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 03 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 7

```
chain = builder.compile()
result = chain.invoke({"topic": "Python 프로그래밍"}, config=lf_config)
print(f"초안: {result['draft']}")
print(f"개선: {result['improved']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 04 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 9

```
from typing import Annotated, TypedDict
import operator


class ParallelState(TypedDict):
    text: str
    analyses: Annotated[list[str], operator.add]


def analyze_tone(state: ParallelState) -> dict:
    r = model.invoke(
        f"한 문장으로 다음의 어조를 설명해주세요: {state['text']}",
        config=lf_config
    )
    return {
        "analyses": [f"어조: {r.content}"]
    }


def analyze_complexity(state: ParallelState) -> dict:
    r = model.invoke(
        f"한 문장으로 다음의 복잡도를 평가해주세요: {state['text']}",
```

04_workflows 코드 04 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 9

```
        config=lf_config
    )
    return {
        "analyses": [f"복잡도: {r.content}"]
    }


def synthesize(state: ParallelState) -> dict:
    return {
        "analyses": [
            f"종합: {len(state['analyses'])}개 분석 완료"
        ]
    }


builder = StateGraph(ParallelState)

builder.add_node("tone", analyze_tone)
builder.add_node("complexity", analyze_complexity)
builder.add_node("synthesize", synthesize)

# START에서 두 분석 노드로 병렬 분기
```

04_workflows 코드 04 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 9

```
builder.add_edge(START, "tone")
builder.add_edge(START, "complexity")

# 두 노드 완료 후 합성
builder.add_edge("tone", "synthesize")
builder.add_edge("complexity", "synthesize")
builder.add_edge("synthesize", END)

parallel_graph = builder.compile()

result = parallel_graph.invoke(
    {
        "text": "LangGraph는 강력한 에이전트 워크플로를 가능하게 합니다.",
        "analyses": []
    },
    config=lf_config
)

for a in result["analyses"]:
    print(f"  {a}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 05 (1/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 11

```
from pydantic import BaseModel, Field
from typing import Literal, TypedDict


class Classification(BaseModel):
    category: Literal["technical", "business", "casual"]


class RouteState(TypedDict):
    question: str
    category: str
    answer: str


def classify(state: RouteState) -> dict:
    structured = model.with_structured_output(Classification)
    result = structured.invoke(
        f"다음 질문을 분류하세요: {state['question']}",
        config=lf_config
    )
    return {"category": result.category}
```

04_workflows 코드 05 (2/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 11

```
def handle_technical(state: RouteState) → dict:
    r = model.invoke(
        f"기술 전문가로서 답변해주세요: {state['question']}",
        config=lf_config
    )
    return {"answer": r.content}


def handle_business(state: RouteState) → dict:
    r = model.invoke(
        f"비즈니스 어드바이저로서 답변해주세요: {state['question']}",
        config=lf_config
    )
    return {"answer": r.content}


def handle_casual(state: RouteState) → dict:
    r = model.invoke(
        f"가볍게 답변해주세요: {state['question']}",
        config=lf_config
    )
```

04_workflows 코드 05 (3/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 11

```
    return {"answer": r.content}


def route(state: RouteState) -> str:
    return state["category"]


builder = StateGraph(RouteState)

builder.add_node("classify", classify)
builder.add_node("technical", handle_technical)
builder.add_node("business", handle_business)
builder.add_node("casual", handle_casual)

builder.add_edge(START, "classify")
builder.add_conditional_edges("classify", route)

builder.add_edge("technical", END)
builder.add_edge("business", END)
builder.add_edge("casual", END)

router = builder.compile()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 05 (4/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 11

```
result = router.invoke(
    {
        "question": "Python에서 가비지 컬렉션은 어떻게 작동하나요?"
    },
    config=lf_config
)

print(f"카테고리: {result['category']}")
print(f"답변: {result['answer'][:200]}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 06 (1/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 13

```
from langgraph.types import Send
from typing import Annotated, TypedDict
import operator


class OrchestratorState(TypedDict):
    topic: str
    sections: list[str]
    results: Annotated[list[str], operator.add]


class WorkerState(TypedDict):
    section: str


def plan_sections(state: OrchestratorState) -> dict:
    r = model.invoke(
        f"'{state['topic']}'에 대한 기사의 짧은 섹션 제목 3개를 나열해주세요. 한 줄에 하나씩, 번호 없이.",
        config=lf_config
    )

    sections = [
```

04_workflows 코드 06 (2/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 13

```
        s.strip()
        for s in r.content.strip().split("\n")
        if s.strip()
    ][:3]

    return {"sections": sections}


def assign_workers(state: OrchestratorState) -> list[Send]:
    return [
        Send("worker", {"section": s})
        for s in state["sections"]
    ]


def worker(state: WorkerState) -> dict:
    r = model.invoke(
        f"다음에 대해 한 문장으로 작성해주세요: {state['section']}",
        config=lf_config
    )

    return {
```

04_workflows 코드 06 (3/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 13

```
        "results": [
            f"## {state['section']}\n{r.content}"
        ]
    }


builder = StateGraph(OrchestratorState)

builder.add_node("plan", plan_sections)
builder.add_node("worker", worker)

builder.add_edge(START, "plan")

builder.add_conditional_edges(
    "plan",
    assign_workers,
    ["worker"]
)

builder.add_edge("worker", END)

orchestrator = builder.compile()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 06 (4/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 13

```
result = orchestrator.invoke(
    {
        "topic": "머신러닝",
        "sections": [],
        "results": []
    },
    config=lf_config
)

for r in result["results"]:
    print(r)
    print()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

04_workflows 코드 07 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 15

```
class EvalState(TypedDict):
    task: str
    draft: str
    feedback: str
    is_good: bool
    iterations: int

def generate(state: EvalState) → dict:
    if state.get("feedback"):
        prompt = f"피드백을 반영하여 개선해주세요.\n원본: {state['draft']}\n피드백: {state['feedback']}"
    else:
        prompt = f"다음에 대한 한 문장 슬로건을 작성해주세요: {state['task']}"
    r = model.invoke(prompt, config=lf_config)
    return {"draft": r.content, "iterations": state.get("iterations", 0) + 1}

def evaluate(state: EvalState) → dict:
    r = model.invoke(f"이 슬로건을 1-10으로 평가하고 간단한 피드백을 주세요: '{state['draft']}'", config=lf_config)
    content = r.content
    is_good = any(f"{n}/10" in content for n in range(8, 11))
    return {"feedback": content, "is_good": is_good}

def should_retry(state: EvalState) → str:
```

04_workflows 코드 07 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

04_workflows.ipynb

cell 15

```
    if state["is_good"] or state["iterations"] >= 3:
        return END
    return "generate"

builder = StateGraph(EvalState)
builder.add_node("generate", generate)
builder.add_node("evaluate", evaluate)

builder.add_edge(START, "generate")
builder.add_edge("generate", "evaluate")
builder.add_conditional_edges("evaluate", should_retry, ["generate", END])

optimizer = builder.compile()
result = optimizer.invoke({"task": "Python 학습 플랫폼"}, config=lf_config)
print(f"최종 초안 ({result['iterations']}번 반복 후): {result['draft']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

05_agents.ipynb

05_agents.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/05_agents.ipynb

05_agents 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agents.ipynb cell 4

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agents 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agents.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agents 코드 03 (1/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agents.ipynb

cell 7

```python
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage, AnyMessage

@tool
def add(a: int, b: int) -> int:
    """두 수를 더합니다."""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """두 수를 곱합니다."""
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """a를 b로 나눕니다."""
    return a / b

tools = [add, multiply, divide]
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agents 코드 03 (2/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agents.ipynb cell 7

```
print("모델에 바인딩된 도구:")
for t in tools:
    print(f"  - {t.name}: {t.description}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agents 코드 04 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

05_agents.ipynb

cell 9

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated, Literal
import operator

class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]

SYSTEM_PROMPT = "당신은 유용한 수학 어시스턴트입니다. 제공된 도구를 사용하여 계산하세요."

def llm_node(state: AgentState) -> dict:
    response = model_with_tools.invoke(
        [SystemMessage(content=SYSTEM_PROMPT)] + state["messages"],
        config=lf_config,
)
    return {"messages": [response]}

def tool_node(state: AgentState) -> dict:
    results = []
    for tc in state["messages"][-1].tool_calls:
        tool_fn = tools_by_name[tc["name"]]
        result = tool_fn.invoke(tc["args"], config=lf_config)
        results.append(ToolMessage(content=str(result), tool_call_id=tc["id"]))
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agents 코드 04 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

05_agents.ipynb

cell 9

```
    return {"messages": results}

def should_continue(state: AgentState) -> Literal["tools", "__end__"]:
    last = state["messages"][-1]
    return "tools" if last.tool_calls else END

builder = StateGraph(AgentState)
builder.add_node("llm", llm_node)
builder.add_node("tools", tool_node)
builder.add_edge(START, "llm")
builder.add_conditional_edges("llm", should_continue, ["tools", END])
builder.add_edge("tools", "llm")

agent = builder.compile()

result = agent.invoke({"messages": [HumanMessage(content="(7 * 8) + 12는 얼마인가요?")]}, config=lf_config)
print("에이전트 답변:", result["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agents 코드 05

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

05_agents.ipynb

cell 11

```
print("에이전트 실행 흐름:")
print("=" * 60)
for chunk in agent.stream(
    {"messages": [HumanMessage(content="100 / 4를 계산한 다음 3을 곱하면 얼마인가요?")]},
    stream_mode="updates",
    config=lf_config,
):
    for node_name, update in chunk.items():
        print(f"\n[{node_name}]")
        for msg in update.get("messages", []):
            if hasattr(msg, 'tool_calls') and msg.tool_calls:
                for tc in msg.tool_calls:
                    print(f"  도구 호출: {tc['name']}({tc['args']})")
            elif hasattr(msg, 'content') and msg.content:
                print(f"  {msg.content[:200]}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

05_agents 코드 06 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

05_agents.ipynb cell 13

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import add_messages

@task
def call_model(messages: list) → object:
    return model_with_tools.invoke(
        [SystemMessage(content=SYSTEM_PROMPT)] + messages,
        config=lf_config,
)

@task
def execute_tool(tool_call: dict) → ToolMessage:
    tool_fn = tools_by_name[tool_call["name"]]
    result = tool_fn.invoke(tool_call["args"], config=lf_config)
    return ToolMessage(content=str(result), tool_call_id=tool_call["id"])

@entrypoint(checkpointer=InMemorySaver())
def functional_agent(messages: list) → list:
    response = call_model(messages).result()

    while response.tool_calls:
```

05_agents 코드 06 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

05_agents.ipynb

cell 13

```
        tool_results = [execute_tool(tc).result() for tc in response.tool_calls]
        messages = add_messages(messages, [response] + tool_results)
        response = call_model(messages).result()

    return add_messages(messages, response)

config = {"configurable": {"thread_id": "func-agent-1"}}
result = functional_agent.invoke(
    [HumanMessage(content="15 * 4를 계산한 다음 20을 더하면 얼마인가요?")],
    {**config, **lf_config}
)
print("Functional 에이전트 답변:", result[-1].content)
```

05_agents 코드 07

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

05_agents.ipynb

cell 15

```
from langgraph.checkpoint.memory import InMemorySaver

agent_with_memory = builder.compile(checkpointer=InMemorySaver())

config = {"configurable": {"thread_id": "math-session"}}

r1 = agent_with_memory.invoke(
    {"messages": [HumanMessage(content="6 * 7은 얼마인가요?")]},
    {**config, **lf_config}
)
print("턴 1:", r1["messages"][-1].content)

r2 = agent_with_memory.invoke(
    {"messages": [HumanMessage(content="이제 그 결과를 2로 나누세요.")]},
    {**config, **lf_config}
)
print("턴 2:", r2["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

06_persistence_and_memory.ipynb

06_persistence_and_memory.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · `03_langgraph/06_persistence_and_memory.ipynb`

06_persistence_and_memory 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_persistence_and_memory.ipynb

cell 4

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
# docs/langgraph 패치 기준 canonical 모델 ID
model = ChatOpenAI(model="gpt-5.4-mini")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_persistence_and_memory.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 03 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 7

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from langchain.messages import HumanMessage

def chatbot(state: MessagesState) -> dict:
    response = model.invoke(state["messages"], config=lf_config)
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

# 체크포인터와 함께 컴파일
checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "session-1"}}

# 대화 1
r1 = graph.invoke({"messages": [HumanMessage(content="제 이름은 앨리스입니다.")]}, {**config, **lf_config})
print("턴 1:", r1["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 03 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 7

```
# 대화 2 — 이전 대화를 기억
r2 = graph.invoke({"messages": [HumanMessage(content="제 이름이 뭐였죠?")]}, {**config, **lf_config})
print("턴 2:", r2["messages"][-1].content)
```

06_persistence_and_memory 코드 04

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_persistence_and_memory.ipynb

cell 9

```
state = graph.get_state(config)
print(f"스레드: {config['configurable']['thread_id']}")
print(f"메시지 수: {len(state.values['messages'])}")
print(f"체크포인트 ID: {state.config['configurable']['checkpoint_id']}")
print(f"다음 노드: {state.next}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 05

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_persistence_and_memory.ipynb

cell 11

```
print("상태 이력 (최신순):")
for i, snapshot in enumerate(graph.get_state_history(config)):
    msg_count = len(snapshot.values.get("messages", []))
    print(f"  [{i}] 체크포인트={snapshot.config['configurable']['checkpoint_id'][:20]}... 메시지={msg_count}")
    if i >= 4:
        print("  ... (생략)")
        break
```

06_persistence_and_memory 코드 06

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_persistence_and_memory.ipynb

cell 13

```
from langchain.messages import AIMessage

# 상태에 메시지 직접 추가
graph.update_state(config, {
    "messages": [AIMessage(content="(시스템 노트: 사용자는 한국어 응답을 선호합니다.)")]
})

# 수정된 상태로 계속 대화
r3 = graph.invoke({"messages": [HumanMessage(content="재미있는 사실을 알려주세요.")]}, {**config, **lf_config})
print("업데이트 후:", r3["messages"][-1].content[:200])
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 07

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

06_persistence_and_memory.ipynb

cell 15

```
config_b = {"configurable": {"thread_id": "session-2"}}
r = graph.invoke({"messages": [HumanMessage(content="제 이름을 아시나요?")]}, {**config_b, **lf_config})
print("다른 스레드:", r["messages"][-1].content)
print("→ 다른 thread_id이므로 이전 대화를 알 수 없음")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 08

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

06_persistence_and_memory.ipynb

cell 17

```
from langgraph.store.memory import InMemoryStore

store = InMemoryStore()

# 데이터 저장
store.put(("users",), "alice", {"favorite_color": "blue", "city": "Seoul"})
store.put(("users",), "bob", {"favorite_color": "red", "city": "Tokyo"})

# 데이터 조회
alice = store.get(("users",), "alice")
print(f"Alice: {alice.value}")

# 검색
results = store.search(("users",))
print(f"\n전체 사용자 ({len(results)}명):")
for item in results:
    print(f"  {item.key}: {item.value}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 09 (1/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 19

```
import uuid
from dataclasses import dataclass
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.store.memory import InMemoryStore
from langgraph.runtime import Runtime
from langchain.messages import HumanMessage

# --- Context schema: typed namespace for user-level data ---
@dataclass
class Context:
    user_id: str

# --- Store and checkpointer ---
memory_store = InMemoryStore()
memory_checkpointer = InMemorySaver()

# --- Node that reads/writes long-term memory via runtime.context + runtime.store ---
def chatbot_with_memory(state: MessagesState, runtime: Runtime[Context]):
    """runtime.context.user_id로 namespace를 만들고 장기 메모리를 읽고 씁니다."""
    user_id = runtime.context.user_id
    namespace = (user_id, "memories")
```

06_persistence_and_memory 코드 09 (2/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 19

```
    # 이 사용자의 기존 메모리를 조회
    existing = runtime.store.search(namespace)
    memory_text = "\n".join(f"- {item.value['data']}" for item in existing)

    # 메모리에서 시스템 컨텍스트 구성
    system_msg = "당신은 유용한 어시스턴트입니다."
    if memory_text:
        system_msg += f"\n\n사용자 메모리:\n{memory_text}"

    messages = [{"role": "system", "content": system_msg}] + state["messages"]
    response = model.invoke(messages, config=lf_config)

    # 사용자가 자신에 대해 언급한 새로운 사실을 저장
    user_msg = state["messages"][-1].content.lower()
    if any(kw in user_msg for kw in ["좋아", "사랑", "이름은", "선호", "즐겨"]):
        runtime.store.put(namespace, str(uuid.uuid4()), {"data": state["messages"][-1].content})

    return {"messages": [response]}

# --- Build graph with context_schema ---
builder = StateGraph(MessagesState, context_schema=Context)
```

06_persistence_and_memory 코드 09 (3/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 19

```
builder.add_node("chatbot", chatbot_with_memory)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

graph_with_store = builder.compile(
    checkpointer=memory_checkpointer,
    store=memory_store,  # ⟵ store를 그래프에 연결
)

# --- Thread 1: user shares preferences ---
config_1 = {"configurable": {"thread_id": "store-thread-1"}}
r1 = graph_with_store.invoke(
    {"messages": [HumanMessage(content="저는 등산과 사진 찍기를 좋아합니다.")]},
    {**config_1, **lf_config},
    context=Context(user_id="user-alice"),
)
print("스레드 1:", r1["messages"][-1].content[:150])

# --- Thread 2 (different thread, same user): memories persist ---
config_2 = {"configurable": {"thread_id": "store-thread-2"}}
r2 = graph_with_store.invoke(
    {"messages": [HumanMessage(content="제 취미가 뭔가요?")]},
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 09 (4/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 19

```
    {**config_2, **lf_config},
    context=Context(user_id="user-alice"),
)
print("\n스레드 2 (새 스레드, 같은 사용자):", r2["messages"][-1].content[:150])

# --- Verify stored memories ---
stored = memory_store.search(("user-alice", "memories"))
print(f"\nuser-alice의 저장된 메모리 ({len(stored)}개):")
for item in stored:
    print(f"  {item.key[:8]}...: {item.value}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 10 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 21

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.messages.utils import trim_messages, count_tokens_approximately
from langchain.messages import HumanMessage, AIMessage, RemoveMessage

# ============================================================
# Part 1: trim_messages — LLM 호출 전에 메시지 잘라내기
# ============================================================

def chatbot_with_trim(state: MessagesState) -> dict:
    """토큰 예산에 맞게 메시지를 잘라낸 후 LLM을 호출합니다."""
    trimmed = trim_messages(
        state["messages"],
        strategy="last",
        token_counter=count_tokens_approximately,
        max_tokens=80,
        start_on="human",
        end_on=("human", "tool"),
    )
    print(f"  원본 메시지: {len(state['messages'])}개, 잘라낸 후: {len(trimmed)}개")
    response = model.invoke(trimmed, config=lf_config)
    return {"messages": [response]}
```

06_persistence_and_memory 코드 10 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 21

```
builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot_with_trim)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

trim_checkpointer = InMemorySaver()
trim_graph = builder.compile(checkpointer=trim_checkpointer)
trim_config = {"configurable": {"thread_id": "trim-demo"}}

# Build up a long conversation
for i, msg in enumerate(["안녕하세요, 제 이름은 앨리스입니다.", "저는 서울에 살고 있습니다.", "저는 엔지니어로 일하고 있습니다.", "제 소개 내용을 정리해 주세요."]):
    print(f"턴 {i+1}: {msg}")
    r = trim_graph.invoke({"messages": [HumanMessage(content=msg)]}, {**trim_config, **lf_config})
    print(f"  AI: {r['messages'][-1].content[:120]}\n")

# Verify full history is preserved in checkpoint despite trimming
full_state = trim_graph.get_state(trim_config)
print(f"체크포인트의 총 메시지 수: {len(full_state.values['messages'])}")
print("→ trim_messages는 LLM 호출 시에만 잘라내고, 체크포인트에는 전체 이력이 보존됩니다")
```

06_persistence_and_memory 코드 11 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 22

```python
# ============================================================
# Part 2: RemoveMessage — 체크포인트에서 메시지 영구 삭제
# ============================================================
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.checkpoint.memory import InMemorySaver
from langchain.messages import HumanMessage, RemoveMessage

def chatbot_node(state: MessagesState) -> dict:
    response = model.invoke(state["messages"], config=lf_config)
    return {"messages": [response]}

def delete_old_messages(state: MessagesState) -> dict:
    """최근 2개 메시지만 유지하고 나머지를 체크포인트에서 삭제합니다."""
    messages = state["messages"]
    if len(messages) > 2:
        to_delete = messages[:-2]
        print(f"  체크포인트에서 {len(to_delete)}개의 오래된 메시지 삭제 중")
        return {"messages": [RemoveMessage(id=m.id) for m in to_delete]}
    return {"messages": []}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot_node)
```

06_persistence_and_memory 코드 11 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 22

```
builder.add_node("cleanup", delete_old_messages)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", "cleanup")
builder.add_edge("cleanup", END)

rm_checkpointer = InMemorySaver()
rm_graph = builder.compile(checkpointer=rm_checkpointer)
rm_config = {"configurable": {"thread_id": "remove-demo"}}

# Run several turns
for msg_text in ["안녕하세요! 저는 Bob입니다.", "저는 요리를 좋아합니다.", "제 이름이 뭐였죠?"]:
    r = rm_graph.invoke({"messages": [HumanMessage(content=msg_text)]}, {**rm_config, **lf_config})
    state = rm_graph.get_state(rm_config)
    print(f"사용자: {msg_text}")
    print(f"  AI: {r['messages'][-1].content[:120]}")
    print(f"  체크포인트 내 메시지 수: {len(state.values['messages'])}\n")

print("→ RemoveMessage로 오래된 메시지를 체크포인트에서 영구적으로 삭제하여 저장 공간을 절약합니다")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 12 (1/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 24

```
import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

# --- State ---
class PipelineState(TypedDict):
    data: str
    log: Annotated[list[str], operator.add]

# Track how many times step_3 has been called across invocations
attempt_count = {"step_3": 0}

# --- Nodes ---
def step_1(state: PipelineState) -> dict:
    """Step 1: 데이터 수집 — 항상 성공합니다."""
    print("  >> step_1 실행됨 (데이터 수집)")
    return {
        "data": "수집된 원시 데이터",
        "log": ["step_1: 데이터 수집 완료"],
    }
```

06_persistence_and_memory 코드 12 (2/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 24

```
def step_2(state: PipelineState) → dict:
    """Step 2: 데이터 분석 — 항상 성공합니다."""
    print("  >> step_2 실행됨 (분석)")
    return {
        "data": state["data"] + " → 분석 완료",
        "log": ["step_2: 데이터 분석 완료"],
    }

def step_3(state: PipelineState) → dict:
    """Step 3: 외부 API 호출 — 첫 번째 시도에서 실패하고, 재시도 시 성공합니다."""
    attempt_count["step_3"] += 1
    print(f"  >> step_3 실행됨 (API 호출, 시도 {attempt_count['step_3']})")
    if attempt_count["step_3"] == 1:
        raise ConnectionError("시뮬레이션된 API 오류!")
    return {
        "data": state["data"] + " → API 결과 수신",
        "log": ["step_3: API 호출 성공"],
    }

# --- Build graph ---
builder = StateGraph(PipelineState)
```

06_persistence_and_memory 코드 12 (3/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 24

```
builder.add_node("step_1", step_1)
builder.add_node("step_2", step_2)
builder.add_node("step_3", step_3)
builder.add_edge(START, "step_1")
builder.add_edge("step_1", "step_2")
builder.add_edge("step_2", "step_3")
builder.add_edge("step_3", END)

durable_checkpointer = InMemorySaver()
durable_graph = builder.compile(checkpointer=durable_checkpointer)

config = {"configurable": {"thread_id": "durable-1"}}

# --- First invocation: step_3 will fail ---
print("═══ 1차 호출: step_3에서 실패 예상 ═══")
try:
    durable_graph.invoke({"data": "", "log": []}, {**config, **lf_config})
except ConnectionError as e:
    print(f"  !! 오류 발생: {e}")

# Check checkpoint state after failure
state_after_fail = durable_graph.get_state(config)
```

06_persistence_and_memory 코드 12 (4/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

06_persistence_and_memory.ipynb

cell 24

```
print(f"\n실패 후 체크포인트:")
print(f"  다음 실행할 노드: {state_after_fail.next}")
print(f"  현재까지 로그: {state_after_fail.values.get('log', [])}")

# --- Second invocation: resumes from step_3 only ---
print("\n═══ 2차 호출: 마지막 체크포인트에서 재개 ═══")
result = durable_graph.invoke(None, {**config, **lf_config})
print(f"\n최종 결과:")
print(f"  데이터: {result['data']}")
print(f"  로그: {result['log']}")
print("\n→ step_1, step_2는 재실행되지 않고 step_3만 재실행되었습니다 (durable execution)")
```

06_persistence_and_memory 코드 13 (1/3)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_persistence_and_memory.ipynb

cell 26

```
# ============================================================================
# 프로덕션 체크포인터 — API 레퍼런스 (실제 실행은 의존 DB가 있을 때만)
# ============================================================================
# 모든 체크포인터는 BaseCheckpointSaver 인터페이스를 구현하므로
# 그래프 측 코드는 동일하고 compile(checkpointer=...) 한 줄만 바뀝니다.

reference_code = r'''
# --- PostgreSQL (sync) ---
from langgraph.checkpoint.postgres import PostgresSaver

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresSaver.from_conn_string(DB_URI) as checkpointer:
    checkpointer.setup()  # 최초 1회 — 스키마 생성
    graph = builder.compile(checkpointer=checkpointer)

# --- PostgreSQL (async) — 비동기 그래프 실행에 사용 ---
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

async with AsyncPostgresSaver.from_conn_string(DB_URI) as checkpointer:
    await checkpointer.setup()
    graph = builder.compile(checkpointer=checkpointer)
    await graph.ainvoke(..., config={"configurable": {"thread_id": "1"}})
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 13 (2/3)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_persistence_and_memory.ipynb cell 26

```
# --- MongoDB ---
from langgraph.checkpoint.mongodb import MongoDBSaver

with MongoDBSaver.from_conn_string("localhost:27017") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)

# --- Redis ---
from langgraph.checkpoint.redis import RedisSaver

with RedisSaver.from_conn_string("redis://localhost:6379") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)

# --- Oracle ---
from langgraph.checkpoint.oracle import OracleSaver

DB_URI = "oracle://user:password@localhost:1521/?service_name=FREEPDB1"
with OracleSaver.from_conn_string(DB_URI) as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)
'''

print(reference_code)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 13 (3/3)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

06_persistence_and_memory.ipynb

cell 26

```
print("패턴 요약:")
print("  1) from_conn_string(DB_URI) 컨텍스트 매니저 진입")
print("  2) 최초 1회 setup() 호출 → 스키마/인덱스 생성")
print("  3) compile(checkpointer=...) 로 그래프에 연결")
print("  4) 동기/비동기 변형: PostgresSaver vs AsyncPostgresSaver, RedisSaver vs AsyncRedisSaver, ...")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

06_persistence_and_memory 코드 14 (1/2)

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

06_persistence_and_memory.ipynb

cell 28

```
# ============================================================================
# 프로덕션 Store — API 레퍼런스
# ============================================================================
store_reference = r'''
# --- PostgresStore ---
from langgraph.store.postgres import PostgresStore

DB_URI = "postgresql://postgres:postgres@localhost:5442/postgres?sslmode=disable"
with PostgresStore.from_conn_string(DB_URI) as store:
    store.setup()
    graph = builder.compile(store=store)

# --- RedisStore ---
from langgraph.store.redis import RedisStore

with RedisStore.from_conn_string("redis://localhost:6379") as store:
    graph = builder.compile(store=store)

# --- OracleStore (vector search 내장) ---
from langgraph.store.oracle import OracleStore

DB_URI = "oracle://user:password@localhost:1521/?service_name=FREEPDB1"
```

06_persistence_and_memory 코드 14 (2/2)

이 코드는 장기 메모리 Store 또는 저장 계층 사용을 보여줍니다.

06_persistence_and_memory.ipynb

cell 28

```
with OracleStore.from_conn_string(DB_URI) as store:
    graph = builder.compile(store=store)

# --- Semantic search 옵션 (모든 store 공통) ---
from langchain.embeddings import init_embeddings
from langgraph.store.memory import InMemoryStore

store = InMemoryStore(
    index={
        "embed": init_embeddings("openai:text-embedding-3-small"),
        "dims": 1536,
        "fields": ["food_preference", "$"],
    }
)
memories = store.search(namespace, query="What does the user like to eat?", limit=3)
'''
print(store_reference)
print("setup() 한 번이면 끝 — 이후 노드 코드는 runtime.store.put/search/aput/asearch 그대로.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

07_streaming.ipynb

07_streaming.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/07_streaming.ipynb

07_streaming 코드 01 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

07_streaming.ipynb

cell 4

```python
from dotenv import load_dotenv
load_dotenv()
from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.messages import HumanMessage, SystemMessage, ToolMessage, AnyMessage
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict, Annotated, Literal
import operator

# docs/langgraph 패치 기준 canonical 모델 ID
model = ChatOpenAI(model="gpt-5.4-mini")

@tool
def search(query: str) -> str:
    """정보를 검색합니다."""
    return f"'{query}' 검색 결과: LangGraph는 상태 기반 에이전트를 구축하는 프레임워크입니다."

tools = [search]
tools_by_name = {t.name: t for t in tools}
model_with_tools = model.bind_tools(tools)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 01 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

07_streaming.ipynb

cell 4

```
class AgentState(TypedDict):
    messages: Annotated[list[AnyMessage], operator.add]

def llm_node(state: AgentState) → dict:
    return {"messages": [model_with_tools.invoke(
        [SystemMessage(content="당신은 유용한 어시스턴트입니다.")] + state["messages"],
        config=lf_config,
)]}

def tool_node(state: AgentState) → dict:
    results = []
    for tc in state["messages"][-1].tool_calls:
        result = tools_by_name[tc["name"]].invoke(tc["args"], config=lf_config)
        results.append(ToolMessage(content=str(result), tool_call_id=tc["id"]))
    return {"messages": results}

def should_continue(state: AgentState) → Literal["tools", "__end__"]:
    return "tools" if state["messages"][-1].tool_calls else END

builder = StateGraph(AgentState)
builder.add_node("llm", llm_node)
builder.add_node("tools", tool_node)
```

07_streaming 코드 01 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

07_streaming.ipynb

cell 4

```
builder.add_edge(START, "llm")
builder.add_conditional_edges("llm", should_continue, ["tools", END])
builder.add_edge("tools", "llm")
agent = builder.compile(checkpointer=InMemorySaver())

print("스트리밍 데모를 위한 에이전트 준비 완료")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

07_streaming.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON — project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")
# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

07_streaming 코드 03

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_streaming.ipynb

cell 8

```
config = {"configurable": {"thread_id": "stream-values"}}
print("═══ stream_mode='values' ═══")
for state in agent.stream(
    {"messages": [HumanMessage(content="LangGraph 정보를 검색해 주세요")]},
    {**config, **lf_config}, stream_mode="values",
):
    msg = state["messages"][-1]
    print(f"  [{type(msg).__name__}] {str(msg.content)[:100]}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 04

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_streaming.ipynb

cell 10

```python
config = {"configurable": {"thread_id": "stream-updates"}}
print("═══ stream_mode='updates' ═══")
for chunk in agent.stream(
    {"messages": [HumanMessage(content="LangGraph 정보를 검색해 주세요")]},
    {**config, **lf_config}, stream_mode="updates",
):
    for node_name, update in chunk.items():
        print(f"\n  [{node_name}]")
        for msg in update.get("messages", []):
            if hasattr(msg, 'tool_calls') and msg.tool_calls:
                for tc in msg.tool_calls:
                    print(f"    Tool: {tc['name']}({tc['args']})")
            elif msg.content:
                print(f"    {msg.content[:150]}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 05

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_streaming.ipynb

cell 12

```
config = {"configurable": {"thread_id": "stream-messages"}}
print("═══ stream_mode='messages' ═══")
print("응답: ", end="")
for msg, metadata in agent.stream(
    {"messages": [HumanMessage(content="LangGraph란 무엇인가요? 간단히 답해 주세요.")]},
    {**config, **lf_config}, stream_mode="messages",
):
    if msg.content and metadata.get("langgraph_node") == "llm":
        print(msg.content, end="", flush=True)
print()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 06

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_streaming.ipynb

cell 14

```
config = {"configurable": {"thread_id": "stream-multi"}}
print("═══ 여러 스트리밍 모드 동시 사용 ═══")
for event in agent.stream(
    {"messages": [HumanMessage(content="Python을 검색해 주세요")]},
    {**config, **lf_config}, stream_mode=["updates", "messages"],
):
    kind = event[0] if isinstance(event, tuple) else "unknown"
    print(f"  [{kind}] 수신됨")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 07 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

07_streaming.ipynb

cell 16

```python
from langgraph.config import get_stream_writer

# --- 커스텀 스트리밍을 사용하는 간단한 그래프 정의 ---
class CustomState(TypedDict):
    topic: str
    result: str

def research_node(state: CustomState) -> dict:
    """research_node: get_stream_writer()로 중간 진행 상황을 스트리밍합니다."""
    writer = get_stream_writer()

    # 1단계: 검색 시작 알림
    writer({"step": 1, "status": "검색 중", "detail": f"'{state['topic']}' 검색 중 ..."})

    # 2단계: 분석 중 알림
    writer({"step": 2, "status": "분석 중", "detail": "검색 결과 분석 중 ..."})

    # 3단계: 완료 알림
    writer({"step": 3, "status": "완료", "detail": "조사 완료."})

    return {"result": f"'{state['topic']}'에 대한 조사가 완료되었습니다."}
```

07_streaming 코드 07 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

07_streaming.ipynb

cell 16

```
custom_builder = StateGraph(CustomState)
custom_builder.add_node("research", research_node)
custom_builder.add_edge(START, "research")
custom_builder.add_edge("research", END)
custom_graph = custom_builder.compile()

# --- stream_mode="custom" 으로 실행 ---
print("═══ stream_mode='custom' ═══")
for chunk in custom_graph.stream(
    {"topic": "LangGraph 스트리밍"},
    stream_mode="custom",
    config=lf_config,
):
    print(f"  커스텀 이벤트: {chunk}")

print("\n--- updates + custom 동시 사용 ---")
for mode, chunk in custom_graph.stream(
    {"topic": "LangGraph 스트리밍"},
    stream_mode=["updates", "custom"],
    config=lf_config,
):
    if mode == "custom":
```

07_streaming 코드 07 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

07_streaming.ipynb

cell 16

```
        print(f"  [custom]  {chunk}")
    else:
        print(f"  [updates] {chunk}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 08 (1/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_streaming.ipynb

cell 18

```
# 앞서 만든 agent에 version="v2"를 그대로 적용
config = {"configurable": {"thread_id": "stream-v2"}}
print("═══ version='v2' + multi-mode ═══")
for chunk in agent.stream(
    {"messages": [HumanMessage(content="LangGraph 정보를 검색해 주세요")]},
    {**config, **lf_config},
    stream_mode=["updates", "messages"],
    version="v2",
):
    # v1에서는 (mode, payload) tuple — v2는 항상 StreamPart dict
    kind = chunk["type"]
    ns   = chunk["ns"]
    data = chunk["data"]

    if kind == "updates":
        for node, upd in data.items():
            msgs = upd.get("messages", [])
            if msgs:
                m = msgs[-1]
                tag = "tool_call" if getattr(m, 'tool_calls', None) else type(m).__name__
                print(f"  [updates/{node}] {tag}")
    elif kind == "messages":
```

07_streaming 코드 08 (2/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_streaming.ipynb

cell 18

```
        msg, meta = data
        if msg.content and meta.get("langgraph_node") == "llm":
            # 토큰 단위 출력
            print(msg.content, end="", flush=True)
print()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 09

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

07_streaming.ipynb

cell 22

```
# chunk_position == "last" 라이브 데모 — 앞서 만든 agent에 그대로 적용
config = {"configurable": {"thread_id": "stream-chunk-position"}}
print("=== chunk_position 감지 (v2 + messages) ===")
print("응답: ", end="")
for chunk in agent.stream(
    {"messages": [HumanMessage(content="LangGraph가 뭔지 한 문장으로만 알려줘.")]},
    {**config, **lf_config},
    stream_mode="messages",
    version="v2",
):
    if chunk["type"] != "messages":
        continue
    msg, meta = chunk["data"]
    if not msg.content or meta.get("langgraph_node") != "llm":
        continue
    print(msg.content, end="", flush=True)
    if meta.get("chunk_position") == "last":
        print("\n  [last chunk 도착 — 메시지 스트리밍 완료]")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 10

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

07_streaming.ipynb

cell 24

```
# GraphOutput 라이브 데모 — agent에 invoke(..., version="v2")
config = {"configurable": {"thread_id": "invoke-v2"}}
result = agent.invoke(
    {"messages": [HumanMessage(content="LangGraph 정보를 검색해 주세요")]},
    {**config, **lf_config},
    version="v2",
)

print(f"type(result):    {type(result).__name__}")
print(f"interrupts:      {len(result.interrupts)}개")
print(f"메시지 수:        {len(result.value['messages'])}")
print(f"마지막 응답:      {result.value['messages'][-1].content[:120]}")

# dict 스타일 접근 (호환용, deprecated)
print(f"\ndict 스타일 호환: result['messages'] 길이={len(result['messages'])}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

07_streaming 코드 11 (1/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

07_streaming.ipynb

cell 26

```
# Event Streaming v3 패턴 — 로컬 설치 버전과 무관하게 안전하게 예시를 출력합니다.
from importlib.metadata import version

print("설치된 langgraph:", version("langgraph"))
print("필요 버전: langgraph ⩾ 1.2.0")

example = r'''
# 1) 기본 사용
stream = graph.stream_events(
    {"messages": [{"role": "user", "content": "42 * 17은?"}]},
    version="v3",
)

for message in stream.messages:
    for token in message.text:
        print(token, end="", flush=True)

for snapshot in stream.values:
    print(snapshot)

final_state = stream.output
```

07_streaming 코드 11 (2/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

07_streaming.ipynb

cell 26

```
# 2) interrupt 이후 재개 — checkpointer + thread_id 필요
from langgraph.types import Command

stream = graph.stream_events(input_data, version="v3")
for message in stream.messages:
    print(message.text)

if stream.interrupted:
    print(stream.interrupts)
    stream = graph.stream_events(
        Command(resume={"decisions": [{"type": "approve"}]}),
        version="v3",
    )
'''
print(example)
```

CODE

08_interrupts_and_time_travel.ipynb

08_interrupts_and_time_travel.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · `03_langgraph/08_interrupts_and_time_travel.ipynb`

08_interrupts_and_time_travel 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

08_interrupts_and_time_travel.ipynb

cell 4

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI

# docs/langgraph 패치 기준 canonical 모델 ID
model = ChatOpenAI(model="gpt-5.4-mini")
print("모델 준비 완료")
```

08_interrupts_and_time_travel 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

08_interrupts_and_time_travel.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

08_interrupts_and_time_travel 코드 03 (1/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 7

```
from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict


class ReviewState(TypedDict):
    document: str
    approved: bool
    final_result: str


def draft_document(state: ReviewState) -> dict:
    return {
        "document": f"초안: {state.get('document', '주제')}에 대한 중요 문서"
    }


def human_review(state: ReviewState) -> dict:
    # 사람의 승인을 기다립니다
    decision = interrupt(
        {
```

08_interrupts_and_time_travel 코드 03 (2/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 7

```
            "document": state["document"],
            "question": "이 문서를 승인하시겠습니까? (yes/no)"
        }
    )

    return {
        "approved": decision == "yes"
    }


def finalize(state: ReviewState) -> dict:
    if state["approved"]:
        return {
            "final_result": f"승인됨: {state['document']}"
        }

    return {
        "final_result": f"거절됨: {state['document']}"
    }


builder = StateGraph(ReviewState)
```

08_interrupts_and_time_travel 코드 03 (3/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 7

```
builder.add_node("draft", draft_document)
builder.add_node("review", human_review)
builder.add_node("finalize", finalize)

builder.add_edge(START, "draft")
builder.add_edge("draft", "review")
builder.add_edge("review", "finalize")
builder.add_edge("finalize", END)

graph = builder.compile(
    checkpointer=InMemorySaver()
)

config = {
    "configurable": {
        "thread_id": "review-1"
    }
}

# Step 1: 실행 → review 노드에서 중단
result = graph.invoke(
```

08_interrupts_and_time_travel 코드 03 (4/4)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 7

```
    {
        "document": "AI 안전성"
    },
    {**config, **lf_config}
)

print("Step 1 - review에서 중단됨")

state = graph.get_state(config)

print(f"  다음 노드: {state.next}")
print(f"  인터럽트 값: {state.tasks}")
```

08_interrupts_and_time_travel 코드 04

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 9

```python
# Step 2: 승인하여 재개

result = graph.invoke(
    Command(resume="yes"),
    {**config, **lf_config}
)

print(f"Step 2 - 승인으로 재개됨")
print(f"  결과: {result['final_result']}")
```

08_interrupts_and_time_travel 코드 05 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 11

```
from langgraph.func import entrypoint, task

@task
def generate_proposal(topic: str) -> str:
    response = model.invoke(f"다음에 대한 한 문장 제안을 작성해주세요: {topic}", config=lf_config)
    return response.content

@entrypoint(checkpointer=InMemorySaver())
def proposal_workflow(topic: str) -> dict:
    proposal = generate_proposal(topic).result()

    # 사람의 승인 대기
    approval = interrupt({
        "proposal": proposal,
        "action": "승인 또는 거절하세요"
    })

    return {
        "proposal": proposal,
        "approved": approval,
    }
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 05 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 11

```
config = {"configurable": {"thread_id": "proposal-1"}}

# 실행 → 중단
result = proposal_workflow.invoke("재생 에너지", {**config, **lf_config})
print("제안 워크플로 중단됨")

# 재개
result = proposal_workflow.invoke(Command(resume="승인"), {**config, **lf_config})
print(f"최종: {result}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 06 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 14

```
from typing import Literal, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver

class ApprovalState(TypedDict):
    action_details: str
    outcome: str

def approval_node(state: ApprovalState) -> Command[Literal["proceed", "cancel"]]:
    decision = interrupt({
        "question": "Approve this action?",
        "details": state["action_details"],
    })
    return Command(goto="proceed" if decision else "cancel")

def proceed(state: ApprovalState) -> dict:
    return {"outcome": f"실행됨: {state['action_details']}"}

def cancel(state: ApprovalState) -> dict:
    return {"outcome": f"취소됨: {state['action_details']}"}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 06 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 14

```
b = StateGraph(ApprovalState)
b.add_node("approval", approval_node)
b.add_node("proceed", proceed)
b.add_node("cancel", cancel)
b.add_edge(START, "approval")
b.add_edge("proceed", END)
b.add_edge("cancel", END)
approval_graph = b.compile(checkpointer=InMemorySaver())

# --- 승인 케이스 ---
cfg_yes = {"configurable": {"thread_id": "approval-yes"}}
out = approval_graph.invoke({"action_details": "결제 처리 $100"}, {**cfg_yes, **lf_config}, version="v2")
print(f"[승인 요청] interrupts={len(out.interrupts)}, value={out.interrupts[0].value}")
out = approval_graph.invoke(Command(resume=True), {**cfg_yes, **lf_config}, version="v2")
print(f"  resume=True  → {out.value['outcome']}")

# --- 거부 케이스 ---
cfg_no = {"configurable": {"thread_id": "approval-no"}}
approval_graph.invoke({"action_details": "결제 처리 $999"}, {**cfg_no, **lf_config}, version="v2")
out = approval_graph.invoke(Command(resume=False), {**cfg_no, **lf_config}, version="v2")
print(f"  resume=False → {out.value['outcome']}")
```

08_interrupts_and_time_travel 코드 07 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 16

```
class ReviewEditState(TypedDict):
    topic: str
    generated_text: str

def generate(state: ReviewEditState) -> dict:
    return {"generated_text": f"AI 초안: {state['topic']}에 대한 짧은 글입니다."}

def review_edit(state: ReviewEditState) -> dict:
    edited = interrupt({
        "instruction": "Review and edit this content",
        "content": state["generated_text"],
    })
    return {"generated_text": edited}

b = StateGraph(ReviewEditState)
b.add_node("generate", generate)
b.add_node("review_edit", review_edit)
b.add_edge(START, "generate")
b.add_edge("generate", "review_edit")
b.add_edge("review_edit", END)
review_graph = b.compile(checkpointer=InMemorySaver())
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 07 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 16

```
cfg = {"configurable": {"thread_id": "review-edit-1"}}
out = review_graph.invoke({"topic": "LangGraph"}, {**cfg, **lf_config}, version="v2")
print(f"[검토 요청] 원본: {out.interrupts[0].value['content']}")

# 사람이 텍스트를 편집해서 resume
out = review_graph.invoke(
    Command(resume="사람이 수정한 글: LangGraph로 상태 기반 워크플로를 만들 수 있다."),
    {**cfg, **lf_config},
    version="v2",
)
print(f"  resume=str  → state.generated_text = {out.value['generated_text']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 08 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 18

```
from langchain.tools import tool

@tool
def send_email(to: str, subject: str, body: str) → str:
    """Send an email to a recipient (sample tool with interrupt)."""
    response = interrupt({
        "action": "send_email",
        "to": to,
        "subject": subject,
        "body": body,
        "message": "Approve sending this email?",
    })

    if response.get("action") == "approve":
        # 사람이 일부 필드만 편집했을 수 있음 → 원본 값과 덮어쓰기
        final_to      = response.get("to", to)
        final_subject = response.get("subject", subject)
        final_body    = response.get("body", body)
        return f"Email sent to {final_to} (subject={final_subject!r})"
    return "Email cancelled by user"

# tool을 그래프 노드로 직접 감싸 데모
```

08_interrupts_and_time_travel 코드 08 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 18

```
class ToolState(TypedDict):
    args: dict
    result: str

def email_node(state: ToolState) -> dict:
    result = send_email.invoke(state["args"])
    return {"result": result}

b = StateGraph(ToolState)
b.add_node("email", email_node)
b.add_edge(START, "email")
b.add_edge("email", END)
tool_graph = b.compile(checkpointer=InMemorySaver())

cfg = {"configurable": {"thread_id": "tool-interrupt-1"}}
out = tool_graph.invoke(
    {"args": {"to": "alice@example.com", "subject": "Hi", "body": "Hello!"}},
    {**cfg, **lf_config},
    version="v2",
)
print(f"[Tool 승인 요청] {out.interrupts[0].value['message']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 08 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 18

```
# 승인 + subject 편집을 동시에
out = tool_graph.invoke(
    Command(resume={"action": "approve", "subject": "Updated subject"}),
    {**cfg, **lf_config},
    version="v2",
)
print(f"  resume=dict → {out.value['result']}")
```

08_interrupts_and_time_travel 코드 09 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb cell 20

```
class AgeState(TypedDict):
    age: int

def get_age_node(state: AgeState) -> dict:
    prompt = "나이를 입력하세요 (양의 정수)."
    while True:
        answer = interrupt(prompt)
        if isinstance(answer, int) and answer > 0:
            break
        prompt = f"'{answer}'은(는) 유효하지 않습니다. 양의 정수를 다시 입력하세요."
    return {"age": answer}

b = StateGraph(AgeState)
b.add_node("ask_age", get_age_node)
b.add_edge(START, "ask_age")
b.add_edge("ask_age", END)
age_graph = b.compile(checkpointer=InMemorySaver())

cfg = {"configurable": {"thread_id": "validation-1"}}
out = age_graph.invoke({"age": 0}, {**cfg, **lf_config}, version="v2")
print(f"  ask    : {out.interrupts[0].value}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 09 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 20

```
# 잘못된 값 — 음수
out = age_graph.invoke(Command(resume=-5), {**cfg, **lf_config}, version="v2")
print(f"  -5     : {out.interrupts[0].value}")

# 잘못된 값 — 문자열
out = age_graph.invoke(Command(resume="abc"), {**cfg, **lf_config}, version="v2")
print(f"  'abc'  : {out.interrupts[0].value}")

# 올바른 값
out = age_graph.invoke(Command(resume=30), {**cfg, **lf_config}, version="v2")
print(f"  30     : 통과 → state.age = {out.value['age']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 10 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 23

```
from langgraph.graph import StateGraph, START, END, MessagesState
from langchain.messages import HumanMessage

def chatbot(state: MessagesState) -> dict:
    response = model.invoke(state["messages"], config=lf_config)
    return {"messages": [response]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

graph = builder.compile(checkpointer=InMemorySaver())

config = {"configurable": {"thread_id": "timetravel-1"}}

# 3번의 대화
graph.invoke({"messages": [HumanMessage(content="제가 좋아하는 색은 파랑입니다.")]}, {**config, **lf_config})
graph.invoke({"messages": [HumanMessage(content="제가 좋아하는 음식은 피자입니다.")]}, {**config, **lf_config})
graph.invoke({"messages": [HumanMessage(content="저는 서울에 살고 있습니다.")]}, {**config, **lf_config})

# 히스토리에서 특정 체크포인트 선택
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 10 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 23

```
history = list(graph.get_state_history(config))
print(f"전체 체크포인트 수: {len(history)}")

# 두 번째 대화 시점으로 되돌아가기 (REPLAY 데모)
target = history[2]  # older checkpoint
target_config = target.config
print(f"\n[Replay] 체크포인트={target_config['configurable']['checkpoint_id'][:20]}...")

# 그 시점에서 새로운 대화 시작
result = graph.invoke(
    {"messages": [HumanMessage(content="제가 좋아하는 음식은 뭔가요?")]},
    {**target_config, **lf_config},
)
print(f"응답: {result['messages'][-1].content[:200]}")
```

08_interrupts_and_time_travel 코드 11 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 25

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class JokeState(TypedDict):
    topic: str
    joke: str

def generate_topic(state: JokeState) -> dict:
    """간단한 토픽 생성 (LLM 호출 비용을 절약하려고 결정적 값을 사용)"""
    return {"topic": "cats"}

def write_joke(state: JokeState) -> dict:
    response = model.invoke(
        f"{state['topic']}에 대한 한 줄 농담을 만들어주세요.",
        config=lf_config,
    )
    return {"joke": response.content}

b = StateGraph(JokeState)
b.add_node("generate_topic", generate_topic)
b.add_node("write_joke", write_joke)
```

08_interrupts_and_time_travel 코드 11 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 25

```
b.add_edge(START, "generate_topic")
b.add_edge("generate_topic", "write_joke")
b.add_edge("write_joke", END)
joke_graph = b.compile(checkpointer=InMemorySaver())

cfg = {"configurable": {"thread_id": "joke-replay"}}
first_run = joke_graph.invoke({"topic": "", "joke": ""}, {**cfg, **lf_config})
print(f"원본:  topic={first_run['topic']!r}, joke={first_run['joke'][:80]!r}")

# write_joke 직전 체크포인트 찾기
history = list(joke_graph.get_state_history(cfg))
before_joke = next(s for s in history if s.next == ("write_joke",))
print(f"\nbefore_joke 체크포인트: next={before_joke.next}, topic={before_joke.values['topic']!r}")

# Replay — generate_topic 결과는 재사용, write_joke만 재실행
replay = joke_graph.invoke(None, {**before_joke.config, **lf_config})
print(f"\nReplay: topic={replay['topic']!r} (재사용), joke={replay['joke'][:80]!r} (재생성)")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 12

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

08_interrupts_and_time_travel.ipynb

cell 27

```
# before_joke 체크포인트에서 topic을 바꾼 fork branch 생성
fork_config = joke_graph.update_state(
    before_joke.config,
    values={"topic": "chickens"},
    as_node="generate_topic",  # 이 update를 generate_topic이 만든 것으로 간주 → write_joke부터 실행
)
print(f"fork_config checkpoint_id: {fork_config['configurable']['checkpoint_id'][:20]}...")

# Fork 실행 — 새 branch에서 write_joke만 실행
fork_result = joke_graph.invoke(None, {**fork_config, **lf_config})
print(f"\nFork:  topic={fork_result['topic']!r} (수정됨), joke={fork_result['joke'][:80]!r}")

# 원본 thread는 그대로 유지
original = joke_graph.get_state(cfg)
print(f"\n원본 유지: topic={original.values['topic']!r}, joke={original.values['joke'][:80]!r}")
print("→ update_state는 thread를 롤백하지 않고, 새 branch를 만들어 갈라집니다.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 13

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

08_interrupts_and_time_travel.ipynb

cell 29

```
# 현재 상태에 정보 추가
from langchain.messages import AIMessage

graph.update_state(config, {
    "messages": [AIMessage(content="(업데이트: 사용자는 고양이도 좋아합니다.)")]
})

result = graph.invoke(
    {"messages": [HumanMessage(content="제 선호에 대해 뭘 알고 있나요?")]},
    {**config, **lf_config}
)
print("상태 업데이트 후:", result["messages"][-1].content[:200])
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 14 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 31

```
# 8.2절에서 만든 리뷰 그래프를 v2로 다시 호출
from langgraph.graph import StateGraph, START, END
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver
from typing import TypedDict

class ReviewState(TypedDict):
    document: str
    approved: bool
    final_result: str

def draft(state):   return {"document": f"초안: {state.get('document','주제')}"}
def review(state):  return {"approved": interrupt({"q": "승인?"}) == "yes"}
def finalize(state):
    verdict = "승인" if state["approved"] else "거절"
    return {"final_result": f"{verdict}: {state['document']}"}

b = StateGraph(ReviewState)
b.add_node("draft", draft); b.add_node("review", review); b.add_node("finalize", finalize)
b.add_edge(START, "draft"); b.add_edge("draft", "review"); b.add_edge("review", "finalize"); b.add_edge("finalize", END)
v2_graph = b.compile(checkpointer=InMemorySaver())
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 14 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

08_interrupts_and_time_travel.ipynb

cell 31

```
cfg = {"configurable": {"thread_id": "v2-review"}}

# Step 1: v2로 실행 → GraphOutput 반환
out = v2_graph.invoke({"document": "AI 안전성"}, {**cfg, **lf_config}, version="v2")

print(f"type: {type(out).__name__}")
print(f"interrupts: {len(out.interrupts)}개")

# v2는 인터럽트 분기가 한 줄
if out.interrupts:
    for iq in out.interrupts:
        print(f"  질문: {iq.value}")
else:
    print(f"  완료 state: {out.value}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

08_interrupts_and_time_travel 코드 15

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

08_interrupts_and_time_travel.ipynb cell 32

```
# Step 2: resume도 v2로 — 최종 state는 .value
out = v2_graph.invoke(Command(resume="yes"), {**cfg, **lf_config}, version="v2")

print(f"interrupts: {len(out.interrupts)}개")
print(f"최종: {out.value['final_result']}")

# v1 호환용 dict-스타일 접근도 여전히 동작 (deprecated)
# 마이그레이션 중 섞여 있어도 깨지지 않음
print(f"dict 접근(호환): {out['final_result']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

09_subgraphs.ipynb

09_subgraphs.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/09_subgraphs.ipynb

09_subgraphs 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_subgraphs.ipynb

cell 4

```python
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
# docs/langgraph 패치 기준 canonical 모델 ID
model = ChatOpenAI(model="gpt-5.4-mini")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

09_subgraphs.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (코드 수정 불필요)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    project = os.environ.get("LANGSMITH_PROJECT", "default")
    print(f"LangSmith tracing ON \u2014 project: {project}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON \u2014 {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

09_subgraphs 코드 03 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 8

```python
from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Annotated
import operator

# ═══ Subgraph 1: Text Analysis ═══
class AnalysisState(TypedDict):
    text: str
    word_count: int
    char_count: int

def count_words(state: AnalysisState) -> dict:
    return {"word_count": len(state["text"].split())}

def count_chars(state: AnalysisState) -> dict:
    return {"char_count": len(state["text"])}

analysis_builder = StateGraph(AnalysisState)
analysis_builder.add_node("words", count_words)
analysis_builder.add_node("chars", count_chars)
analysis_builder.add_edge(START, "words")
analysis_builder.add_edge("words", "chars")
analysis_builder.add_edge("chars", END)
```

09_subgraphs 코드 03 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 8

```
analysis_subgraph = analysis_builder.compile()

# Test subgraph independently
result = analysis_subgraph.invoke({"text": "LangGraph에서 보내는 인사"}, config=lf_config)
print(f"서브그래프 테스트: 단어={result['word_count']}, 문자={result['char_count']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 04 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 10

```
# ═══ Parent Graph: Text Pipeline ═══

class PipelineState(TypedDict):
    text: str
    word_count: int
    char_count: int
    summary: str


def create_summary(state: PipelineState) -> dict:
    return {
        "summary": f"텍스트는 {state['word_count']}개의 단어와 {state['char_count']}개의 문자를 포함합니다."
    }


parent_builder = StateGraph(PipelineState)

# Add subgraph as a node (shared keys: text, word_count, char_count)
parent_builder.add_node("analyze", analysis_subgraph)
parent_builder.add_node("summarize", create_summary)

parent_builder.add_edge(START, "analyze")
```

09_subgraphs 코드 04 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 10

```
parent_builder.add_edge("analyze", "summarize")
parent_builder.add_edge("summarize", END)

pipeline = parent_builder.compile()

result = pipeline.invoke(
    {
        "text": "LangGraph는 강력한 상태 기반 에이전트 워크플로를 가능하게 합니다"
    },
    config=lf_config
)

print(f"요약: {result['summary']}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 05 (1/7)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 12

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END


# ── Step 1: 서브그래프 정의 (자체 상태 스키마: SubState) ──
class SubState(TypedDict):
    log_entry: str
    sentiment: str
    confidence: float


def analyze_sentiment(state: SubState) → dict:
    """간단한 규칙 기반 감성 분석"""
    text = state["log_entry"].lower()

    positive_words = {
        "good", "great", "excellent", "happy", "love", "amazing", "wonderful"
    }

    negative_words = {
        "bad", "terrible", "awful", "hate", "poor", "horrible", "sad"
    }
```

09_subgraphs 코드 05 (2/7)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 12

```
    pos = sum(1 for w in text.split() if w in positive_words)
    neg = sum(1 for w in text.split() if w in negative_words)

    total = pos + neg

    if total == 0:
        return {"sentiment": "neutral", "confidence": 0.5}

    elif pos > neg:
        return {"sentiment": "positive", "confidence": round(pos / total, 2)}

    else:
        return {"sentiment": "negative", "confidence": round(neg / total, 2)}


sub_builder = StateGraph(SubState)

sub_builder.add_node("analyze_sentiment", analyze_sentiment)
sub_builder.add_edge(START, "analyze_sentiment")
sub_builder.add_edge("analyze_sentiment", END)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 05 (3/7)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 12

```
sentiment_subgraph = sub_builder.compile()


# 서브그래프 단독 테스트
sub_result = sentiment_subgraph.invoke(
    {"log_entry": "The product is great and amazing"},
    config=lf_config
)

print(
    f"[서브그래프 단독 테스트] sentiment={sub_result['sentiment']}, confidence={sub_result['confidence']}"
)


# ── Step 2: 부모 그래프 정의 (다른 상태 스키마: ParentState) ──
class ParentState(TypedDict):
    customer_name: str
    feedback_text: str
    analysis_result: str


# ── Step 3: 래퍼 함수 — 상태 변환의 핵심 ──
```

09_subgraphs 코드 05 (4/7)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 12

```
def call_sentiment_subgraph(state: ParentState) -> dict:
    """
    래퍼 함수가 하는 일:

    1) 부모 상태의 'feedback_text' -> 서브그래프의 'log_entry'로 변환
    2) 서브그래프 실행
    3) 서브그래프 출력을 부모 상태의 'analysis_result'로 매핑
    """

    # Parent -> Subgraph: 상태 변환
    subgraph_input = {
        "log_entry": state["feedback_text"]
    }

    # 서브그래프 실행
    subgraph_output = sentiment_subgraph.invoke(
        subgraph_input,
        config=lf_config
    )

    # Subgraph -> Parent: 결과 매핑
    result_str = (
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 05 (5/7)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 12

```
        f"{subgraph_output['sentiment']} "
        f"(confidence: {subgraph_output['confidence']})"
    )

    return {"analysis_result": result_str}


def generate_report(state: ParentState) -> dict:
    """최종 보고서 생성"""
    report = (
        f"[보고서] {state['customer_name']}: "
        f"{state['analysis_result']}"
    )

    return {"analysis_result": report}


# ── Step 4: 부모 그래프 구성 ──
parent_builder = StateGraph(ParentState)

parent_builder.add_node("sentiment_analysis", call_sentiment_subgraph)
parent_builder.add_node("report", generate_report)
```

09_subgraphs 코드 05 (6/7)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 12

```
parent_builder.add_edge(START, "sentiment_analysis")
parent_builder.add_edge("sentiment_analysis", "report")
parent_builder.add_edge("report", END)

parent_graph = parent_builder.compile()


# 실행
result = parent_graph.invoke(
    {
        "customer_name": "Alice",
        "feedback_text": "The product is great and the service was excellent",
    },
    config=lf_config
)

print(f"\n[최종 결과] {result['analysis_result']}")


# 다른 입력으로 추가 테스트
result2 = parent_graph.invoke(
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 05 (7/7)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 12

```
    {
        "customer_name": "Bob",
        "feedback_text": "The experience was terrible and the quality is poor",
    },
    config=lf_config
)

print(f"[최종 결과] {result2['analysis_result']}")
```

09_subgraphs 코드 06 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 14

```
from langchain.messages import HumanMessage, AnyMessage
from langgraph.graph import MessagesState

# Subgraph: Translation Agent
def translator(state: MessagesState) -> dict:
    response = model.invoke(
        state["messages"] + [HumanMessage(content="위 내용을 한국어로 번역해 주세요. 번역만 답해 주세요.")],
        config=lf_config,
)
    return {"messages": [response]}

translator_builder = StateGraph(MessagesState)
translator_builder.add_node("translate", translator)
translator_builder.add_edge(START, "translate")
translator_builder.add_edge("translate", END)
translator_subgraph = translator_builder.compile()

# Subgraph: Summarization Agent
def summarizer(state: MessagesState) -> dict:
    response = model.invoke(
        state["messages"] + [HumanMessage(content="위 내용을 한 문장으로 요약해 주세요.")],
        config=lf_config,
```

09_subgraphs 코드 06 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 14

```
)
    return {"messages": [response]}

summarizer_builder = StateGraph(MessagesState)
summarizer_builder.add_node("summarize", summarizer)
summarizer_builder.add_edge(START, "summarize")
summarizer_builder.add_edge("summarize", END)
summarizer_subgraph = summarizer_builder.compile()

# Parent: Translate-then-Summarize Pipeline
parent_builder = StateGraph(MessagesState)
parent_builder.add_node("translator", translator_subgraph)
parent_builder.add_node("summarizer", summarizer_subgraph)
parent_builder.add_edge(START, "translator")
parent_builder.add_edge("translator", "summarizer")
parent_builder.add_edge("summarizer", END)

pipeline = parent_builder.compile()

result = pipeline.invoke({
    "messages": [HumanMessage(content="LangGraph는 LLM을 사용하여 상태 기반 멀티 액터 애플리케이션을 구축하는 프레임워크입니다.")]
}, config=lf_config)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 06 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 14

```
print("최종 출력:", result["messages"][-1].content)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 07 (1/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

09_subgraphs.ipynb

cell 16

```
print("══ v1: (namespace, data) 튜플 ══")
for chunk in pipeline.stream(
    {"messages": [HumanMessage(content="Python은 범용 프로그래밍 언어입니다.")]},
    stream_mode="updates",
    subgraphs=True,
    config=lf_config,
):
    if isinstance(chunk, tuple) and len(chunk) == 2:
        ns, update = chunk
        for node, data in update.items():
            prefix = f"[{'/'.join(ns)}]" if ns else "[root]"
            print(f"  {prefix} {node}")

print("\n══ v2: StreamPart dict ══")
for chunk in pipeline.stream(
    {"messages": [HumanMessage(content="Python은 범용 프로그래밍 언어입니다.")]},
    stream_mode="updates",
    subgraphs=True,
    version="v2",
    config=lf_config,
):
    # chunk["type"] == "updates", chunk["ns"] = (), 또는 ("translator:<uuid>",) ...
```

09_subgraphs 코드 07 (2/2)

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

09_subgraphs.ipynb

cell 16

```
ns = chunk["ns"]
data = chunk["data"]
prefix = f"[{'/'.join(ns)}]" if ns else "[root]"
for node, _ in data.items():
    print(f"  {prefix} type={chunk['type']!r}, node={node}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 08 (1/3)

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

09_subgraphs.ipynb

cell 18

```python
# 3가지 checkpointer 모드 — API 레퍼런스 (create_agent를 subagent로 쓰는 예)
from langchain.agents import create_agent
from langchain.tools import tool
from langgraph.checkpoint.memory import MemorySaver

@tool
def fruit_info(fruit_name: str) -> str:
    """Look up fruit info."""
    return f"Info about {fruit_name}"

# ═══ 1) Per-Invocation (기본) ═══
# subagent에 checkpointer 미지정 → 부모로부터 상속, 호출마다 fresh
fruit_subagent_per_invocation = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
    prompt="You are a fruit expert.",
)

# ═══ 2) Per-Thread (stateful) ═══
fruit_subagent_per_thread = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
```

09_subgraphs 코드 08 (2/3)

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

09_subgraphs.ipynb

cell 18

```
    prompt="You are a fruit expert.",
    checkpointer=True,  # ← 핵심
)

# ═══ 3) Stateless ═══
fruit_subagent_stateless = create_agent(
    model="gpt-5.4-mini",
    tools=[fruit_info],
    prompt="You are a fruit expert.",
    checkpointer=False,  # ← interrupt 불가
)

# ═══ Per-Thread를 tool로 노출할 때 — 병렬 호출 차단 ═══
reference_code = r'''
from langchain.agents.middleware import ToolCallLimitMiddleware

@tool
def ask_fruit_expert(question: str) → str:
    """Ask the fruit expert."""
    response = fruit_subagent_per_thread.invoke(
        {"messages": [{"role": "user", "content": question}]},
    )
```

09_subgraphs 코드 08 (3/3)

이 코드는 LangChain create_agent 조립과 실행을 보여줍니다.

09_subgraphs.ipynb

cell 18

```
    return response["messages"][-1].content

agent = create_agent(
    model="gpt-5.4-mini",
    tools=[ask_fruit_expert],
    middleware=[
        ToolCallLimitMiddleware(tool_name="ask_fruit_expert", run_limit=1),
    ],
    checkpointer=MemorySaver(),
)
'''
print(reference_code)
print("3가지 모드 객체 생성 완료:",
      type(fruit_subagent_per_invocation).__name__,
      type(fruit_subagent_per_thread).__name__,
      type(fruit_subagent_stateless).__name__)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 09 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 20

```
from langgraph.graph import MessagesState, StateGraph
from langchain.agents import create_agent
from langchain.tools import tool


@tool
def veggie_info(veggie_name: str) -> str:
    """Look up veggie info."""
    return f"Info about {veggie_name}"


def create_sub_agent(model: str, *, name: str, **kwargs):
    """고유 노드 이름으로 agent를 감싸 namespace를 격리합니다."""
    agent = create_agent(model=model, name=name, **kwargs)
    return (
        StateGraph(MessagesState)
        .add_node(name, agent)        # 고유 이름 → 안정적 namespace
        .add_edge("__start__", name)
        .compile()
    )
```

© 주식회사 똑똑한청년들 · 배움 에이아이

09_subgraphs 코드 09 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

09_subgraphs.ipynb

cell 20

```
# 서로 다른 두 per-thread subagent — namespace가 격리됨
fruit_agent_isolated = create_sub_agent(
    "gpt-5.4-mini",
    name="fruit_agent",
    tools=[fruit_info],
    prompt="You are a fruit expert.",
    checkpointer=True,
)

veggie_agent_isolated = create_sub_agent(
    "gpt-5.4-mini",
    name="veggie_agent",
    tools=[veggie_info],
    prompt="You are a veggie expert.",
    checkpointer=True,
)

print("두 per-thread subagent가 서로 다른 namespace로 격리되었습니다.")
print("  - fruit_agent: namespace prefix = ('fruit_agent:<uuid>',)")
print("  - veggie_agent: namespace prefix = ('veggie_agent:<uuid>',)")
print("→ 두 agent의 checkpoint state가 충돌하지 않습니다.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

10_production.ipynb

10_production.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/10_production.ipynb

10_production 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

10_production.ipynb

cell 4

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

10_production 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

10_production.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (LANGSMITH_* env vars 사용)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

10_production 코드 03

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

10_production.ipynb

cell 7

```
import json

config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./agent.py:graph"
    },
    "env": ".env"
}
print("langgraph.json 예시:")
print(json.dumps(config, indent=2))
print()
print("명령어:")
print("  $ pip install 'langgraph-cli[inmem]'")
print("  $ langgraph dev  # http://localhost:2024에서 로컬 서버 시작")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

10_production 코드 04 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

10_production.ipynb

cell 13

```
from langgraph.graph import StateGraph, START, END
from typing import TypedDict

# Graph to test
class TestState(TypedDict):
    input: str
    output: str


def process(state: TestState) -> dict:
    return {"output": state["input"].upper()}


builder = StateGraph(TestState)

builder.add_node("process", process)
builder.add_edge(START, "process")
builder.add_edge("process", END)

graph = builder.compile()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

10_production 코드 04 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

10_production.ipynb

cell 13

```
# Unit tests
def test_process():
    result = graph.invoke({"input": "hello"})

    assert result["output"] == "HELLO", f"HELLO 예상, {result['output']} 반환됨"

    print("  OK test_process")


def test_empty_input():
    result = graph.invoke({"input": ""})

    assert result["output"] == "", f"빈 문자열 예상, {result['output']} 반환됨"

    print("  OK test_empty_input")


print("테스트 실행 중:")

test_process()
test_empty_input()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

10_production 코드 04 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

10_production.ipynb

cell 13

```
print("모든 테스트 통과!")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

10_production 코드 05 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

10_production.ipynb

cell 15

```
from langchain_core.language_models import GenericFakeChatModel
from langchain.messages import AIMessage, HumanMessage, AnyMessage
from langgraph.graph import StateGraph, START, END, MessagesState

# Deterministic fake model
fake_model = GenericFakeChatModel(
    messages=iter(
        [
            AIMessage(content="The answer is 42."),
        ]
    )
)

def chatbot(state: MessagesState) -> dict:
    return {
        "messages": [fake_model.invoke(state["messages"])]
    }

builder = StateGraph(MessagesState)

builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

10_production 코드 05 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

10_production.ipynb

cell 15

```
builder.add_edge("chatbot", END)

test_graph = builder.compile()

result = test_graph.invoke(
    {
        "messages": [HumanMessage(content="테스트")]
    }
)

assert "42" in result["messages"][-1].content

print("GenericFakeChatModel 테스트 통과!")
print(f"  응답: {result['messages'][-1].content}")
```

10_production 코드 06

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

10_production.ipynb

cell 20

```python
import os

tracing = os.environ.get("LANGSMITH_TRACING", "false")
api_key = os.environ.get("LANGSMITH_API_KEY", "")

print("현재 상태:")

print(f"  트레이싱: {'활성화' if tracing == 'true' else '비활성화'}")
print(f"  API 키: {'설정됨' if api_key else '미설정'}")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

11_local_server.ipynb

11_local_server.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/11_local_server.ipynb

11_local_server 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

11_local_server.ipynb

cell 4

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

11_local_server 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

11_local_server.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (LANGSMITH_* env vars 사용)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

11_local_server 코드 03

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

11_local_server.ipynb

cell 7

```python
# LangGraph CLI 설치 명령어
print("═══ pip으로 설치 (Python ≥ 3.11) ═══")
print('  $ pip install -U "langgraph-cli[inmem]"')
print()
print("═══ uv로 설치 ═══")
print('  $ uv add "langgraph-cli[inmem]"')
print()
print("설치 후 확인:")
print("  $ langgraph --version")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

11_local_server 코드 04

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

11_local_server.ipynb

cell 9

```
# 프로젝트 생성 명령어
print("═══ 템플릿으로 새 프로젝트 생성 ═══")
print("  $ langgraph new my-agent --template new-langgraph-project-python")
print()
print("═══ 인터랙티브 메뉴로 생성 ═══")
print("  $ langgraph new my-agent")
print()
print("═══ 생성 후 의존성 설치 ═══")
print("  # pip 사용")
print("  $ cd my-agent && pip install -e .")
print()
print("  # uv 사용")
print("  $ cd my-agent && uv sync")
print()
print("═══ 생성되는 파일 구조 ═══")
print("  my-agent/")
print("  ├── langgraph.json   # 그래프 설정")
print("  ├── .env.example     # 환경 변수 템플릿")
print("  ├── pyproject.toml   # 의존성 정의")
print("  └── src/")
print("      └── agent.py     # 에이전트 코드")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

11_local_server 코드 05

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

11_local_server.ipynb

cell 11

```
import json

# langgraph.json 설정 예시
config = {
    "dependencies": ["."],
    "graphs": {
        "agent": "./src/agent.py:graph"
    },
    "env": ".env"
}

print("langgraph.json 예시:")
print(json.dumps(config, indent=2))
print()
print("주요 필드:")
print('  dependencies: 설치할 패키지 경로 목록')
print('  graphs:       그래프 이름 → 모듈:변수 매핑')
print('  env:          환경 변수 파일 경로')
```

© 주식회사 똑똑한청년들 · 배움 에이아이

11_local_server 코드 06 (1/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

11_local_server.ipynb

cell 15

```
# 비동기 클라이언트 사용 패턴 (서버 실행 중일 때 사용)
print("""from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,
        "agent",
        input={
            "messages": [{
                "role": "human",
                "content": "What is LangGraph?",
            }],
        },
    ):
        print(f"Event type: {chunk.event}...")
        print(chunk.data)

asyncio.run(main())
""")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

11_local_server 코드 06 (2/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

11_local_server.ipynb cell 15

```
print("# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

11_local_server 코드 07

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

11_local_server.ipynb

cell 17

```
# 동기 클라이언트 사용 패턴 (서버 실행 중일 때 사용)
print("""from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,
    "agent",
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Event: {chunk.event}...")
    print(chunk.data)
""")
print("# 위 코드는 langgraph dev 서버가 실행 중일 때 사용합니다.")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

12_durable_execution.ipynb

12_durable_execution.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/12_durable_execution.ipynb

12_durable_execution 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

12_durable_execution.ipynb cell 4

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

12_durable_execution.ipynb cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (LANGSMITH_* env vars 사용)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

12_durable_execution 코드 03

이 코드는 노트북 실습의 실행 단계를 그대로 확인하기 위한 코드입니다.

12_durable_execution.ipynb

cell 10

```
# 문제가 있는 접근 방식: 부수 효과를 직접 호출
print("""# BAD: 부수 효과가 태스크로 감싸지지 않음
def call_api(state: State):
    # 이 API 호출은 재개 시 다시 실행됨!
    result = requests.get(state['url']).text[:100]
    return {"result": result}
""")
print("문제점:")
print("  1. 장애 후 재개 시 API가 다시 호출됨")
print("  2. 비결정적 결과가 달라질 수 있음")
print("  3. 중복 요청으로 부작용 발생 가능")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 04

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

12_durable_execution.ipynb

cell 12

```
# 개선된 접근 방식: @task로 부수 효과 래핑
print("""# GOOD: @task로 부수 효과를 감쌈
from langgraph.func import task

@task
def _make_request(url: str):
    return requests.get(url).text[:100]

def call_api(state: State):
    # 각 요청이 개별 태스크로 실행됨
    requests = [_make_request(url) for url in state['urls']]
    results = [req.result() for req in requests]
    return {"results": results}
""")
print("개선 효과:")
print("  1. 재개 시 체크포인트에서 결과 복원")
print("  2. API 중복 호출 방지")
print("  3. 각 태스크가 독립적으로 추적됨")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 05 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

12_durable_execution.ipynb cell 14

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver


class DocState(TypedDict):
    topic: str
    draft: str
    final: str


def write_draft(state: DocState) -> dict:
    return {"draft": f"Draft about {state['topic']}"}


def finalize(state: DocState) -> dict:
    return {"final": f"Final: {state['draft']}"}


checkpointer = InMemorySaver()

builder = StateGraph(DocState)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 05 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

12_durable_execution.ipynb

cell 14

```
builder.add_node("write_draft", write_draft)
builder.add_node("finalize", finalize)
builder.add_edge(START, "write_draft")
builder.add_edge("write_draft", "finalize")
builder.add_edge("finalize", END)

graph = builder.compile(checkpointer=checkpointer)

# 실행 (thread_id로 실행 추적)
config = {"configurable": {"thread_id": "doc-1"}}
result = graph.invoke({"topic": "LangGraph"}, config)
print("결과:", result)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 06 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

12_durable_execution.ipynb

cell 16

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


@task
def generate_draft(topic: str) -> str:
    return f"Draft about {topic}"


@task
def review_draft(draft: str) -> str:
    return f"Reviewed: {draft}"


func_checkpointer = InMemorySaver()


@entrypoint(checkpointer=func_checkpointer)
def write_document(topic: str) -> str:
    draft = generate_draft(topic).result()
    reviewed = review_draft(draft).result()
    return reviewed
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 06 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

12_durable_execution.ipynb

cell 16

```
config = {"configurable": {"thread_id": "func-1"}}
result = write_document.invoke("Durable Execution", config)
print("결과:", result)
```

12_durable_execution 코드 07 (1/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

12_durable_execution.ipynb cell 18

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver


class PipelineState(TypedDict):
    data: str
    step: int
    result: str


call_count = 0


def step_one(state: PipelineState) -> dict:
    global call_count
    call_count += 1
    print(f"  step_one 실행 (호출 횟수: {call_count})")
    return {"data": state["data"].upper(), "step": 1}


def step_two(state: PipelineState) -> dict:
```

12_durable_execution 코드 07 (2/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

12_durable_execution.ipynb

cell 18

```
    print(f"  step_two 실행")
    return {"result": f"Processed: {state['data']}", "step": 2}


recovery_saver = InMemorySaver()

builder = StateGraph(PipelineState)
builder.add_node("step_one", step_one)
builder.add_node("step_two", step_two)
builder.add_edge(START, "step_one")
builder.add_edge("step_one", "step_two")
builder.add_edge("step_two", END)

pipeline = builder.compile(checkpointer=recovery_saver)

# 첫 번째 실행
config = {"configurable": {"thread_id": "recovery-1"}}
print("═══ 첫 번째 실행 ═══")
result = pipeline.invoke(
    {"data": "hello", "step": 0, "result": ""},
    config
)
```

12_durable_execution 코드 07 (3/3)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

12_durable_execution.ipynb cell 18

```
print(f"결과: {result}")

# 체크포인트 확인
print("\n═══ 체크포인트에서 상태 복원 ═══")
saved = pipeline.get_state(config)
print(f"저장된 상태: {saved.values}")
print(f"step_one 총 호출 횟수: {call_count}")
```

12_durable_execution 코드 08 (1/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

12_durable_execution.ipynb cell 22

```
# LangGraph 1.2 fault tolerance 패턴
from importlib.metadata import version

print("설치된 langgraph:", version("langgraph"))
print("필요 버전: langgraph ≥ 1.2.0")

example = r'''
from langgraph.errors import NodeError
from langgraph.types import Command, RetryPolicy, TimeoutPolicy
from langgraph.runtime import Runtime  # heartbeat 사용 시

def payment_error_handler(state: State, error: NodeError) → Command:
    # retry 모두 소진 후 실행되는 보정 함수
    return Command(
        update={"status": f"compensated: {error.error}"},
        goto="finalize",
    )

builder.add_node(
    "charge_payment",
    charge_payment,
    timeout=TimeoutPolicy(idle_timeout=30),                    # NodeTimeoutError
```

12_durable_execution 코드 08 (2/2)

이 코드는 interrupt와 Command 기반 제어 흐름을 보여줍니다.

12_durable_execution.ipynb

cell 22

```
    retry_policy=RetryPolicy(max_attempts=3, retry_on=ConnectionError),
    error_handler=payment_error_handler,                        # 보정 트랜잭션
)

# 장기 작업 노드: heartbeat로 idle timeout 갱신
async def long_running_node(state: State, runtime: Runtime) -> dict:
    for batch in fetch_batches():
        process(batch)
        runtime.heartbeat()
    return {"result": "done"}

builder.add_node(
    "long_running_node",
    long_running_node,
    timeout=TimeoutPolicy(idle_timeout=30, refresh_on="heartbeat"),
)
'''
print(example)
```

12_durable_execution 코드 09 (1/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

12_durable_execution.ipynb

cell 24

```
# Graceful shutdown 패턴 — langgraph ≥ 1.2
example = r'''
from langgraph.runtime import RunControl, Runtime
from langgraph.errors import GraphDrained

# 1) 호출 측: 외부 시그널(SIGTERM 등)에 drain 요청
control = RunControl()
control.request_drain("sigterm")

try:
    result = graph.invoke(inputs, config, control=control)
except GraphDrained as e:
    print(f"Drained: {e.reason}")
    # checkpoint는 이미 저장됨 — 같은 config로 재개 가능

# 같은 config로 재개
graph.invoke(None, config)

# 2) 노드 내부: drain 신호를 직접 감지
async def my_node(state: State, runtime: Runtime) → dict:
    if runtime.drain_requested:
        return {"status": "skipped"}
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 09 (2/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

12_durable_execution.ipynb cell 24

```
    # ... 일반 처리
    return {"status": "done"}
'''
print(example)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

12_durable_execution 코드 10

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

12_durable_execution.ipynb

cell 26

```
# DeltaChannel 개념 예시 — 실제 실행은 langgraph≥1.2에서 확인하세요.
example = r'''
from typing_extensions import Annotated, TypedDict
from langgraph.channels.delta import DeltaChannel

def append(state: list[str], writes: list[list[str]]) → list[str]:
    return state + [item for batch in writes for item in batch]

class State(TypedDict):
    # append-heavy 메시지/로그 채널에서 checkpoint delta 저장을 고려
    items: Annotated[list[str], DeltaChannel(reducer=append, snapshot_frequency=50)]
'''
print(example)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CODE

13_api_guide_and_pregel.ipynb

13_api_guide_and_pregel.ipynb의 실행 코드 셀을 순서대로 훑습니다. ipynb와 덱만으로 코드 흐름을 복습할 수 있게 구성했습니다.

실습 · 03_langgraph/13_api_guide_and_pregel.ipynb

13_api_guide_and_pregel 코드 01

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb

cell 4

```
from dotenv import load_dotenv
load_dotenv(override=True)
from langchain_openai import ChatOpenAI
model = ChatOpenAI(model="gpt-5.4")
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 02

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb

cell 5

```
# Observability 설정 (선택) - LangSmith 또는 Langfuse
# .env에 키를 설정하거나, 아래 주석을 해제하여 직접 입력하세요.
# os.environ["LANGFUSE_SECRET_KEY"] = "sk-lf-..."
# os.environ["LANGFUSE_PUBLIC_KEY"] = "pk-lf-..."
# os.environ["LANGFUSE_HOST"] = "https://lf.ddok.ai"
import os

# LangSmith: LANGSMITH_TRACING=true 시 자동 활성화 (LANGSMITH_* env vars 사용)
if os.environ.get("LANGSMITH_TRACING", "").lower() == "true":
    os.environ.setdefault("LANGSMITH_API_KEY", os.environ.get("LANGSMITH_API_KEY", ""))
    os.environ.setdefault("LANGSMITH_PROJECT", os.environ.get("LANGSMITH_PROJECT", "default"))
    print(f"LangSmith tracing ON — project: {os.environ['LANGSMITH_PROJECT']}")

# Langfuse: invoke/stream 호출 시 config={"callbacks": [langfuse_handler]} 전달
langfuse_handler = None
if os.environ.get("LANGFUSE_SECRET_KEY"):
    from langfuse.langchain import CallbackHandler
    langfuse_handler = CallbackHandler()
    print(f"Langfuse tracing ON — {os.environ.get('LANGFUSE_HOST', '')}")

# Langfuse config: pass to invoke/stream/batch calls
lf_config = {"callbacks": [langfuse_handler]} if langfuse_handler else {}
```

13_api_guide_and_pregel 코드 03 (1/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

13_api_guide_and_pregel.ipynb

cell 9

```
from typing import TypedDict
from langgraph.constants import START
from langgraph.graph import StateGraph


class Essay(TypedDict):
    topic: str
    content: str | None
    score: float | None


def write_essay(essay: Essay):
    return {"content": f"Essay about {essay['topic']}"}


def score_essay(essay: Essay):
    return {"score": 10}


builder = StateGraph(Essay)
builder.add_node(write_essay)
builder.add_node(score_essay)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 03 (2/2)

이 코드는 StateGraph 상태, 노드, 엣지 조립을 보여줍니다.

13_api_guide_and_pregel.ipynb

cell 9

```
builder.add_edge(START, "write_essay")
builder.add_edge("write_essay", "score_essay")

graph_app = builder.compile()

result = graph_app.invoke({"topic": "LangGraph"})
print("Graph API 결과:", result)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 04 (1/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

13_api_guide_and_pregel.ipynb

cell 11

```
from typing import TypedDict
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver


class EssayResult(TypedDict):
    topic: str
    content: str | None
    score: float | None


@task
def write_essay_func(topic: str) -> str:
    return f"Essay about {topic}"


@task
def score_essay_func(content: str) -> float:
    return 10


func_saver = InMemorySaver()
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 04 (2/2)

이 코드는 Functional API의 task/entrypoint 실행 단위를 보여줍니다.

13_api_guide_and_pregel.ipynb

cell 11

```
@entrypoint(checkpointer=func_saver)
def essay_pipeline(topic: str) -> dict:
    content = write_essay_func(topic).result()
    score = score_essay_func(content).result()
    return {"topic": topic, "content": content, "score": score}


config = {"configurable": {"thread_id": "essay-1"}}
result = essay_pipeline.invoke("LangGraph", config)
print("Functional API 결과:", result)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 05 (1/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb

cell 16

```python
from langgraph.channels import EphemeralValue
from langgraph.pregel import Pregel, NodeBuilder

# 단일 노드: 입력을 두 번 반복
node1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b")
)

app = Pregel(
    nodes={"node1": node1},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
    },
    input_channels=["a"],
    output_channels=["b"],
)

result = app.invoke({"a": "foo"})
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 05 (2/2)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb

cell 16

```
print("Pregel 결과:", result)
# 'foo' + 'foo' = 'foofoo'
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 06 (1/4)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb cell 18

```
from langgraph.channels import (
    EphemeralValue,
    LastValue,
    Topic,
    BinaryOperatorAggregate,
)
from langgraph.pregel import Pregel, NodeBuilder

# --- 1. LastValue: 최신 값만 유지 ---
node_lv = (
    NodeBuilder()
    .subscribe_only("input")
    .do(lambda x: x.upper())
    .write_to("output")
)

app_lv = Pregel(
    nodes={"node": node_lv},
    channels={
        "input": EphemeralValue(str),
        "output": LastValue(str),
    },
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 06 (2/4)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb cell 18

```
    input_channels=["input"],
    output_channels=["output"],
)
print("LastValue:", app_lv.invoke({"input": "hello"}))

# --- 2. Topic: 여러 값을 누적 ---
node_t1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)

node_t2 = (
    NodeBuilder()
    .subscribe_to("b")
    .do(lambda x: x["b"] + x["b"])
    .write_to("c")
)

app_topic = Pregel(
    nodes={"node1": node_t1, "node2": node_t2},
```

13_api_guide_and_pregel 코드 06 (3/4)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb cell 18

```
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": Topic(str, accumulate=True),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("Topic:", app_topic.invoke({"a": "foo"}))

# --- 3. BinaryOperatorAggregate: 리듀서 적용 ---
def reducer(current, update):
    if current:
        return current + " | " + update
    return update

node_b1 = (
    NodeBuilder()
    .subscribe_only("a")
    .do(lambda x: x + x)
    .write_to("b", "c")
)
```

13_api_guide_and_pregel 코드 06 (4/4)

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb cell 18

```
node_b2 = (
    NodeBuilder()
    .subscribe_only("b")
    .do(lambda x: x + x)
    .write_to("c")
)

app_agg = Pregel(
    nodes={"node1": node_b1, "node2": node_b2},
    channels={
        "a": EphemeralValue(str),
        "b": EphemeralValue(str),
        "c": BinaryOperatorAggregate(str, operator=reducer),
    },
    input_channels=["a"],
    output_channels=["c"],
)
print("BinaryOperatorAggregate:", app_agg.invoke({"a": "foo"}))
```

© 주식회사 똑똑한청년들 · 배움 에이아이

13_api_guide_and_pregel 코드 07

이 코드는 실습에 필요한 패키지와 객체를 준비합니다.

13_api_guide_and_pregel.ipynb

cell 20

```
# DeltaChannel 개념 예시 — 실제 실행은 langgraph≥1.2에서 확인하세요.
example = r'''
from typing import Annotated, Sequence
from typing_extensions import TypedDict
from langgraph.channels import DeltaChannel

def list_reducer(state: list, writes: Sequence[list]) → list:
    # (current_state, sequence_of_writes) → new_state
    result = list(state)
    for write in writes:
        result.extend(write)
    return result

class State(TypedDict):
    messages: Annotated[
        list[str],
        DeltaChannel(list_reducer, snapshot_frequency=5),
    ]
'''
print(example)
```

© 주식회사 똑똑한청년들 · 배움 에이아이

CLOSING

LangGraph 심화는 '그래프를 그리는 법'보다 '실행을 통제하는 법'입니다

다음 Deep Agents 실전에서는 이 제어 모델 위에 파일, 서브에이전트, 메모리, 스킬과 선택형 TodoListMiddleware를 얹어 장기 작업 하네스를 만듭니다.